

Evolving Intelligent Network Attack Classifier Under Label Distribution Shift

Miru Kim , Mugon Joe , and Minhae Kwon , *Senior Member, IEEE*

Abstract—The next-generation internet is being reshaped by the growing intelligence and connectivity of artificial intelligence of things (AIoT) devices. This evolution emphasizes the importance of AI-enabled network intrusion detection systems (NIDS) in intelligent network environments. Two major technical challenges in developing such systems are addressing class imbalance and adapting to label distribution shifts after deployment. Class imbalance, caused by the dominance of normal traffic and the scarcity and uneven distribution of attack types, often results in low performance for minority attack type classes. In addition, evolving attack patterns necessitate efficient and continuous model post-training, despite the limited and unlabeled data available in the post-deployment phase. This study proposes a unified training framework consisting of a two-step pre-training scheme and an adaptive post-training scheme. In Step I of pre-training, balanced contrastive pair selection is performed to account for class imbalance, while Step II refines the representations of hard-to-distinguish samples located near decision boundaries using latent features. In the post-training phase, the framework leverages both model outputs and latent representations to generate reliable pseudo-labels for learning. A key insight of this work is that a well-structured latent representation space enables reliable pseudo-labeling during post-training. Extensive simulations across eight online label shift scenarios with three datasets demonstrate that the proposed method achieves up to an 8.6% improvement in accuracy and F1-score over eight state-of-the-art approaches, along with a significant reduction in post-training complexity.

Index Terms—Contrastive learning, label shift, online learning, imbalanced dataset, low-rank adaptation, network intrusion detection systems.

I. INTRODUCTION

THE rapid evolution of AIoT networks has profoundly impacted industries by integrating a diverse array of devices, leading to enhanced efficiency in areas such as automation, real-time monitoring, and management. However, this increased

connectivity simultaneously introduces the frequency of network intrusions [1]. When successful, these intrusions can result in significant privacy breaches and substantial financial losses. To preserve network security, the development of intelligent NIDS capable of accurately classifying attack types is critical to secure highly connected environments [2], [3], [4], [5], [6], [7]. Consequently, attack type classification models have gained considerable attention, and extensive research has focused on improving their effectiveness for intrusion detection [8], [9], [10], [11], [12], [13], [14]. Despite these efforts, three key challenges continue to impede the performance of these models.

The first challenge arises during the pre-training phase before the model is deployed, where the dataset often exhibits a significant class imbalance. In such cases, some major classes have exceedingly more samples than other minor classes, leading to a skewed data distribution [15]. This imbalance causes models to become biased toward the majority class, resulting in poor detection rates for the minority classes, which are often critical intrusion types. Consequently, many studies have explored methods for improving the representation of minority classes in the latent space [16], [17], which focus on enhancing the learning process for these underrepresented classes, e.g., synthetic minority oversampling technique (SMOTE) [18] and contrastive learning. However, existing approaches mainly balance sample distributions without explicitly addressing intrinsically hard-to-distinguish samples, which limits their effectiveness in improving fundamental discrimination performance. In contrast, our method directly tackles this limitation by focusing on performance enhancement for such challenging samples.

Even with well-trained models, dynamic real-world networks present the second challenge: online label shift. As the label distribution of traffic types changes over time, it may diverge from the training data, potentially causing significant drops in the model performance [19]. While model post-training has become necessary, the lack of labeled data in real-time environments poses a considerable obstacle [20]. Recent studies have explored the post-training of models without true labels using techniques such as pseudo-labeling, in which the model generates labels based on predictions from a pre-trained model [21], [22]. However, without proper validation, incorrect pseudo-labels may degrade the model performance during additional training. This highlights the need for confidence measures to assess their reliability, ensuring that only dependable pseudo-labels are used for post-training.

While ensuring pseudo-label reliability is essential, efficiency during the post-training phase is also critical. To minimize the

Received 1 November 2025; revised 4 January 2026; accepted 28 January 2026. Date of publication 3 March 2026; date of current version 19 March 2026. This work was supported by the National Research Foundation of Korea funded by the Korean Government (MSIT) under Grant RS-2025-02214082 and Grant RS-2023-00278812. Recommended for acceptance by Dr. Weiting Zhang. (Miru Kim and Mugon Joe are co-first authors.) (Corresponding author: Minhae Kwon.)

Miru Kim and Minhae Kwon are with the Department of Electrical and Computer Engineering, Sungkyunkwan University, Suwon 16419, Republic of Korea (e-mail: miru.kim@skku.edu; minhae.kwon@skku.edu).

Mugon Joe is with the Department of Intelligent Semiconductors, Soongsil University, Seoul 06978, Republic of Korea (e-mail: mugon@soongsil.ac.kr).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TNSE.2026.3669948>, provided by the authors.

Digital Object Identifier 10.1109/TNSE.2026.3669948

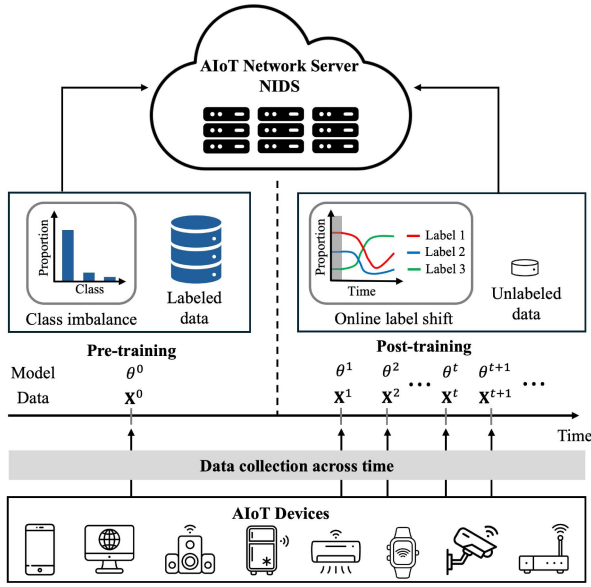


Fig. 1. Overview of the proposed AIoT network attack classification system. The diagram illustrates the processes to address the class imbalance problem in pre-training and efficient post-training with a small amount of unlabeled data over an online label shift environment.

computational burden, additional training must be performed only when necessary [23]. Moreover, with limited data available in the post-training phase, the training must also be sample-efficient. To achieve this, we employ LoRA [24], which reduces the number of training parameters such that the model can be optimized with minimal and enhances sample efficiency without fully updating the model. Nevertheless, conventional LoRA-based methods still perform updates at every time step, leading to unnecessary overhead. In contrast, our scheme adaptively triggers post-training only when the cosine similarity between consecutive outputs indicates substantial changes in distribution, thereby enabling more efficient adaptation without sacrificing accuracy.

To address these challenges, this study proposes a novel framework that jointly integrates three contributions: i) contrastive learning designed to improve performance on hard-to-distinguish samples for class imbalance, ii) reliable pseudo-label generation using latent representations for online label shift, and iii) adaptive LoRA-based post-training triggered by distributional change for efficient adaptation. Together, these components provide a unified solution for robust and efficient model adaptation in dynamic network environments. Fig. 1 illustrates an overview of this system.

Our contributions are summarized as follows.

- We propose a two-stage framework explicitly designed to handle real-world AIoT challenges, including the performance enhancement of minor-but-critical attack classes and evolving label distributions after deployment.
- To tackle class imbalance in the training dataset, we introduce a novel borderline sample refinement technique that improves representations near decision boundaries. This includes inverse probability sampling for selecting

balanced contrastive learning pairs and the use of Mahalanobis distance in the latent space.

- We design an efficient and lightweight LoRA-based post-training mechanism that adaptively triggers model updates only when both high uncertainty and significant label shift are observed. We also propose a pseudo-labeling method based on entropy and distance-based confidence.
- We conduct extensive simulations on three popular NIDS datasets, covering four online label shift scenarios, five ablation studies, and performance comparisons against eight state-of-the-art algorithms. Additionally, we analyze parameter sensitivity, robustness to the imbalance factor, and the impact of the number of post-training samples.

The remainder of this paper is organized as follows. Section II reviews related works, and Section III presents the system setup for NIDS in a label shift environment. Section IV details the proposed pre-training methods, and Section V describes the post-training methods. Section VI presents the simulation setup, and Section VII, and Section VIII provide simulation results for pre-training and post-training, respectively. Finally, Section IX concludes the paper.

II. RELATED WORKS

In this section, we review existing literature on the three key areas relevant to our study. Table I summarizes the comparison between our proposed method and existing approaches, outlining the key challenges and techniques employed in each work.

A. Network Attack Classification

Network attack classification is an essential step in building a NIDS because identifying intrusion types can lead to an appropriate response. A challenge arises from the varying amount of data collected for each intrusion type, resulting in class imbalance. This imbalance can bias models toward majority classes, making it difficult to classify minority classes [25], [26].

Traditional methods for addressing class imbalance typically involve oversampling methods. Random oversampling increases the number of minority class samples to balance the dataset but often leads to overfitting by replicating data without introducing variability [27], [28]. SMOTE generates synthetic samples in the minority class, thereby rebalancing the data distribution to mitigate the dominance of majority classes [18]. Borderline-SMOTE refines this approach by creating synthetic samples near decision boundaries where the classification uncertainty is higher [29]. Another approach is focal loss [30], which modifies the standard cross-entropy loss function to reduce the impact of easily classified samples. Despite their widespread use in network intrusion classification, these methods often fail to yield meaningful diversity in the latent space, resulting in insufficiently discriminative embeddings.

To address these challenges, contrastive learning has emerged as an effective method for constructing well-defined latent spaces for class imbalance. Using label information, the model is trained to bring samples of the same class closer while far apart samples from different classes, which helps reduce the bias toward majority classes [16], [31]. Contrastive learning

TABLE I
COMPARISON OF THE PROPOSED METHOD WITH EXISTING APPROACHES

Papers	Challenges			Solutions		
	Class Imbalance	Online Label Shift	Few Sample	Contrastive Learning	Unsupervised Post-training	LoRA
[27]	✓	✗	✗	✗	✗	✗
[18]	✓	✗	✗	✗	✗	✗
[29]	✓	✗	✗	✗	✗	✗
[30]	✓	✗	✗	✗	✗	✗
[32]	✓	✗	✗	✓	✗	✗
[33]	✓	✗	✗	✓	✗	✗
[35]	✗	✓	✗	✗	✓	✗
[36]	✗	✓	✗	✗	✓	✗
[37]	✗	✗	✓	✗	✗	✓
[38]	✗	✗	✓	✗	✗	✓
This Work	✓	✓	✓	✓	✓	✓

enhances the quality of feature representations, thereby allowing the model to capture subtle differences between classes [32]. Studies on contrastive learning emphasize borderline samples near decision boundaries, which are particularly challenging to classify. By focusing on these difficult cases, contrastive loss functions help the model learn a well-represented feature space. In addition, distance-based criteria are used to identify informative pairs and improve the ability of the model to classify minority classes [33].

The selection of informative pairs remains a key challenge in contrastive learning [34]. The proposed method addresses this issue by prioritizing samples from minority classes. Subsequently, it applies a distance-based criterion to select borderline samples. By refining these processes, the model enhances its ability to distinguish between classes, thereby improving the classification of minor classes.

B. Online Label Shift

Online label shift occurs when the distribution of labels changes over time. This is common in dynamic environments, where intrusion types evolve continuously [39]. A significant challenge arises when the label distribution in the post-training differs from that of the training data, leading to a decline in the model performance.

Several methods have been proposed to address online label shift [35], [36], [40]. The follow the leader (FTL) strategy was extended with the follow the history (FTH) [35] algorithm, which averages the label distributions observed up to the current time step. This method assumes that previous distributions can inform future distributions, making them effective for gradual shifts. For dynamic scenarios, follow the fixed window history (FTFWH) [35] approach adapts more quickly by focusing on recent data within a fixed window. It prioritizes recent observations, offering responsiveness to rapid shifts but can be sensitive to the chosen window size. Another approach, the unbiased online gradient descent (UOGD) [36] algorithm, builds on the classic online gradient descent (OGD) using an unbiased risk estimator, allowing continuous updates without labeled data. UOGD is effective in relatively stable environments

but struggles in highly dynamic environments because of its fixed learning rate and limited adaptability to rapid changes. A significant contribution in this field is the adapting to label shift (ATLAS) [36] algorithm, which uses an online ensemble approach with multiple base learners, each with a different step size. This meta-algorithm dynamically combines learners based on the intensity of the label distribution shifts to achieve optimal dynamic regret bounds without requiring prior knowledge of non-stationarity.

Recent studies have also used pseudo-labeling methods to adapt models when labeled data are scarce. Pseudo-labeling utilizes model predictions as labels for further training, thereby allowing continuous post-training without explicit labels [41]. In [42], a training system refined the predictions by selecting high-confidence pseudo-labels, ensuring that only reliable data were used for post-training. Similarly, [43] proposed a confidence-based training method that adjusts the learning process based on prediction certainty. Moreover, [44] applied pseudo-labeling in online label shift scenarios using model outputs to adapt continuously as label distributions evolved. This approach is effective in dynamic environments, including AIoT networks [21], [41]. However, the quality of pseudo-labels remains crucial because poorly generated labels can harm the model performance, particularly under significant label shifts.

Our system utilizes information from a pre-trained model to align the pseudo-labeling process during post-training. Moreover, we only use pseudo-labels that satisfy the reliability requirements. This approach ensures that the model adapts accurately and maintains robust performance, even when the label distributions shift significantly.

C. Low-Rank Matrix Adaptation

When a pre-trained model is deployed in the real world, additional training is often required to post-train the model and maximize its performance in the test environment. LoRA [45] has emerged as a promising parameter-efficient fine-tuning (PEFT) solution because it reduces the number of trainable parameters and allows for more efficient post-training. LoRA decomposes a large weight matrix into two low-rank matrices and updates

TABLE II
NOTATION TABLE

Notation	Description	Notation	Description
$\mathbf{X}^0$	Pre-training dataset	$\mathbf{x}_i^0$	i -th input in pre-training
$\mathbf{y}^0$	Pre-training labels	y_i^0	Label for $\mathbf{x}_i^0$
$\mathbf{X}^t$	Post-training dataset at time t	$\mathbf{x}_i^t$	i -th input at time t
N^0	Number of pre-training samples	N^t	Number of post-training samples
$\tilde{y}_i^t$	Pseudo-label for $\mathbf{x}_i^t$	$\hat{y}_i^t$	Predicted label for $\mathbf{x}_i^t$
$ \mathcal{C} $	Number of classes	$\mathcal{C}$	Class set
c	Class label	θ^0	Pre-trained model parameters
θ^t	Model parameters at time t	$\theta_{:l}^t$	Model parameters up to the l -th layer
θ_l^t	Model parameters of the l -th layer	$\theta_{:l}^t(\cdot)$	Output of the l -th layer at time t
L	Number of layers	$\bar{L}$	Contrastive representation layer
d_l	Dimension of layer l	$\mathcal{L}_{\text{pre}}$	Pre-training loss
$\mathcal{L}_{\text{post}}$	Post-training loss	$\mathcal{L}_{\text{cross}}$	Cross-entropy loss
$\mathcal{L}_{\text{cont}}$	Contrastive loss	$p_{\mathbf{x}_i^0}(c)$	True class probability
$q_{\mathbf{x}_i^t}(c; \theta^t)$	Predicted probability	T	Total time steps
t	Current time	α^t	Transition parameter
$\Omega^0(c)$	Initial label distribution	$\Omega^t(c)$	Label distribution at time t
$\omega_1(c)$	Initial class distribution	$\omega_2(c)$	Final class distribution
$\mathbf{x}_{a,c}^0$	Anchor sample for class c	$\mathbf{x}_{b,c}^0$	Borderline sample for class c
$y_{a,c}^0$	Anchor label for class c	$y_{b,c}^0$	Borderline label for class c
$D_{\text{MD}}(\cdot)$	Mahalanobis distance	Δ_{MD}	MD difference
$h(\cdot)$	Entropy	$\{A, B\}$	Low-rank matrices
λ	Scaling parameter	η	Learning rate

TABLE III
SUMMARY OF THRESHOLD SYMBOLS AND THEIR CORRESPONDING ROLES

Symbol	Name	Description
ϕ_{ent}	Entropy threshold for post-training	Used to measure the model's confidence based on the entropy of output predictions. When the entropy exceeds this threshold, the model detects a label distribution shift and triggers post-training.
ϕ_{cos}	Cosine similarity threshold	Used to measure the similarity between the current and previous label distributions, assisting in the decision of whether post-training should be activated.
ϕ_{pred}	Entropy threshold for pseudo-labeling	Applied during the pseudo-labeling process to determine whether a prediction is reliable enough to be directly used as a pseudo-label.
ϕ_{MD}	Mahalanobis distance threshold	Used in the pseudo-labeling process to assess the reliability of labels derived from the latent feature representations.

only these low-rank matrices, thereby significantly reducing the number of parameters that require adjustment. This approach is particularly effective in scenarios where post-training data are scarce.

In [38], LoRA was applied to a NIDS, enabling the model to maintain a high detection accuracy even with fewer parameters to be updated. Similarly, [37] demonstrated that LoRA reduces the number of parameters required for model updates while allowing the system to adapt effectively to dynamic environments. Given its ability to reduce the number of parameters requiring updates while maintaining performance, we also incorporate LoRA in the final layer of the pre-trained model to ensure efficient post-training with limited post-training data.

III. SYSTEM SETUP

We consider a NIDS for AIoT systems that includes a traffic type classifier. The NIDS monitors incoming traffic from AIoT devices and uses a classification model to classify traffic types such as normal, attack type 1, attack type 2, etc. The system operates in real-time to quickly detect potential

security threats. Building a high-accuracy classifier is important, as accurate identification of the intrusion type can support actions such as selecting an appropriate mitigation strategy, adjusting security policies, or prioritizing alerts based on the severity or characteristics of the detected attack. A comprehensive notation summary, defining all symbols and parameters associated with the proposed NIDS and the traffic classification process, is presented in Tables II and III.

Suppose that the traffic type classifier at time t employs a neural network model θ^t with L layers. The input of the model at time t is the network traffic stream $\mathbf{X}^t = \{\mathbf{x}_i^t\}_{i=1}^{N^t}$, where $\mathbf{x}_i^t$ denotes i -th sample at time t , which has multiple features, and N^t denotes the number of samples at time t . The output of the model for the input $\mathbf{x}_i^t$ is denoted by $q_{\mathbf{x}_i^t}(c; \theta^t)$, which is the predicted probability over a set of classes $c \in \mathcal{C}$, where $\mathcal{C}$ denotes the set of class indices and $|\mathcal{C}|$ denotes the number of classes. Here, θ^t represents the entire set of parameters of the neural network model at time t . We denote $\theta_{:l}^t(\mathbf{x}_i^t)$ for $1 \leq l \leq L$ as the output of the l -th layer of the model when $\mathbf{x}_i^t$ is the input. Note that θ_l^t is the weight parameter of the l -th layer, whereas $\theta_{:l}^t$ refers to the parameters from the first layer up to the l -th layer.

The model θ^t undergoes two training phases: pre-training with a pre-collected dataset and online post-training using real-time data samples.

First, the model undergoes a pre-training phase at time $t = 0$. The pre-training dataset $(\mathbf{X}^0, \mathbf{y}^0)$ is labeled, containing traffic type information $\mathbf{y}^0 = \{y_i^0\}_{i=1}^{N^0}$ as the true labels corresponding to $\mathbf{X}^0$. The label $y_i^0 \in \mathcal{C}$ denotes the class index for the data sample $\mathbf{x}_i^0$. For each data sample $\mathbf{x}_i^0$, we define the true class probability $p_{\mathbf{x}_i^0}(c)$ using the Dirac delta function as follows.

$$p_{\mathbf{x}_i^0}(c) = \begin{cases} 1, & \text{if } c = y_i^0 \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

The pre-training dataset $(\mathbf{X}^0, \mathbf{y}^0)$ is imbalanced, as the majority of traffic samples are normal, while attack samples are rare. Even among attack classes, the number of samples for each class varies significantly because some attack types are more common than others. Consequently, the label distribution of the pre-training dataset is highly skewed. In this pre-training phase, the model θ^0 is trained in a supervised manner using the true label, i.e.,

$$\theta^0 \leftarrow \theta^0 - \eta \nabla \mathcal{L}_{\text{pre}}(\cdot), \quad (2)$$

where $\mathcal{L}_{\text{pre}}(\cdot)$ denotes the loss function and η is the learning rate for the pre-training phase. The primary challenge in this phase is addressing imbalanced data, which makes a model biased toward majority classes.

Once the model is fully trained, it is deployed in the real world. The model then enters the post-training phase at time $1 \leq t \leq T$, where T is the total time. In this phase, the model is fine-tuned with a few online samples. Model post-training is necessary when a shift in the label distribution occurs, which leads to a higher occurrence of minority class samples. Although the pre-training phase attempts to address the imbalanced dataset, the model may still have low accuracy for minority class samples. These label distribution shifts frequently occur in NIDS because newly developed attack types may be underrepresented in the pre-training dataset but become prevalent once the model is deployed.

The label distribution changes over time in the real world. This shift is modeled as a label distribution, i.e.,

$$\Omega^t(c) = (1 - \alpha^t) \omega_1(c) + \alpha^t \omega_2(c), \quad (3)$$

where $\omega_1(c)$ and $\omega_2(c)$ represent the initial and final class distributions, respectively, and α^t controls the transition.¹ At each time t , it receives an unlabeled dataset $\mathbf{X}^t = \{\mathbf{x}_i^t\}_{i=1}^{N^t}$ consisting of N^t samples. Note that the number of data samples in the post-training phase is considerably smaller than that in the pre-training phase, i.e., $N^t \ll N^0$.

To minimize the computational burden, post-training is activated only when necessary. We first assess whether the post-training condition is satisfied, and if so, proceed with the post-training. However, a significant challenge arises when true

Algorithm 1: Pre-Training Phase.

Require: Dataset $\mathbf{X}^0 = \{\mathbf{x}_i^0\}_{i=1}^{N^0}$, $\mathbf{y}^0 = \{y_i^0\}_{i=1}^{N^0}$, mean vectors $\{\mu_c\}_{c \in \mathcal{C}}$, covariance matrices $\{\Sigma_c\}_{c \in \mathcal{C}}$, borderline threshold ϕ_{border}

Ensure: Trained model θ^0

```

1: // Step I: Imbalance-aware Training
2: for  $i = 1$  to  $N$  do
3:   Select the first sample  $(\mathbf{x}_i^0, y_i^0)$ 
4:   Sample the second sample  $(\mathbf{x}_j^0, y_j^0)$  with  $y_j^0 \sim \Psi^0(c)$ 
5:   Compute  $\mathcal{L}_{\text{pre}}(\mathbf{x}_i^0, \mathbf{x}_j^0, y_i^0, y_j^0; \theta^0)$  in (6)
6:   Update model  $\theta^0 \leftarrow \theta^0 - \eta \nabla \mathcal{L}_{\text{pre}}(\mathbf{x}_i^0, \mathbf{x}_j^0, y_i^0, y_j^0; \theta^0)$ 
7: end for
8: // Step II: Borderline Sample Refinement
9: for  $i = 1$  to  $N$  do
10:  Compute  $D_{\text{MD}}(\mathbf{x}_i^0, y_i^0, \mu_c, \Sigma_c; \theta_{\text{L}}^0)$ 
11:  if  $D_{\text{MD}}(\mathbf{x}_i^0, y_i^0, \mu_c, \Sigma_c; \theta_{\text{L}}^0) > \phi_{\text{border}}$  then
12:    Assign  $(\mathbf{x}_i^0, y_i^0)$  as the borderline sample  $(\mathbf{x}_{b,c}^0, y_{b,c}^0)$ 
13:  end if
14: end for
15: for  $c \in \mathcal{C}$  do
16:  Select anchor sample  $(\mathbf{x}_{a,c}^0, y_{a,c}^0)$  based on (13)
17:  for all borderline samples in class  $c$  do
18:    Compute  $\mathcal{L}_{\text{pre}}(\mathbf{x}_{a,c}^0, \mathbf{x}_{b,c}^0, y_{a,c}^0, y_{b,c}^0; \theta^0)$  in (6)
19:    Update model  $\theta^0 \leftarrow \theta^0 - \eta \nabla \mathcal{L}_{\text{pre}}(\mathbf{x}_{a,c}^0, \mathbf{x}_{b,c}^0, y_{a,c}^0, y_{b,c}^0; \theta^0)$ 
20:  end for
21: end for

```

labels are unavailable during the post-training phase. A pseudo-label $\tilde{y}_i^t$ must be generated to train the model via supervised learning. We denote $\tilde{p}_{\mathbf{x}_i^t}(c)$ as the class probability based on the pseudo-label $\tilde{y}_i^t$, which is similar to (1). Another challenge is that the training method for the post-training phase must be sample-efficient because only a few data samples are available. In this post-training phase, if the post-training condition is satisfied, the model θ^t is updated as follows, where $\mathcal{L}_{\text{post}}(\cdot)$ is the loss function during the post-training phase.

$$\theta^t \leftarrow \theta^t - \eta \nabla \mathcal{L}_{\text{post}}(\cdot) \quad (4)$$

Our goal is to maximize both the classification accuracy and training sample efficiency in this online label distribution shift environment. To achieve this, we propose a contrastive learning-based pre-training method specifically designed to enhance the minority class performance. In addition, we design post-training criteria, a pseudo-labeling method, and a LoRA-based efficient training method that can enhance the model by updating fewer parameters, thereby requiring fewer samples for post-training.

IV. CONTRASTIVE LEARNING BASED PRE-TRAINING

In this section, we introduce the pre-training phase of the proposed system, which consists of two steps: imbalance-aware training and borderline sample refinement. The pre-training phase aims to improve the accuracy of the minor class with better feature representation by incorporating both cross-entropy

¹In Section VI-B, we provide four types of α^t settings, including linear, square, sine, and Bernoulli.

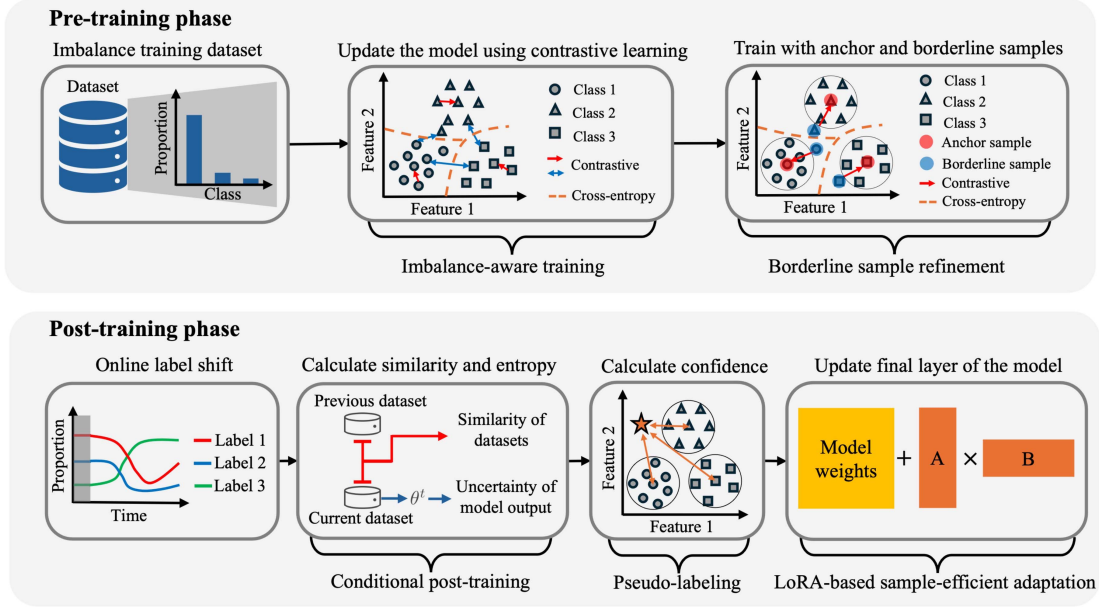


Fig. 2. Overview of the proposed system. The figure illustrates the pre-training and post-training phases.

and contrastive loss functions. During this phase, the model alternates between two steps for each epoch. These two steps are illustrated in Fig. 2, and the pseudocode is presented in Algorithm 1.

A. Step I: Imbalance-Aware Training

In this step, the goal is to address the class imbalance problem using a contrastive learning system. To employ contrastive learning, we first select a pair of data samples $(\mathbf{x}_i^0, \mathbf{x}_j^0, y_i^0, y_j^0)$. The first sample $(\mathbf{x}_i^0, y_i^0)$ is determined sequentially, i.e., $\forall \mathbf{x}_i^0 \in \mathbf{X}^0, \forall y_i^0 \in \mathbf{Y}^0$, leading to follow the label distribution of the pre-training $\Omega^0(c)$, i.e., $y_i^0 \sim \Omega^0(c)$. The second sample $(\mathbf{x}_j^0, y_j^0)$ is selected according to the probability of $\Psi^0(c)$, i.e., $y_j^0 \sim \Psi^0(c)$, which provides a higher selection probability for the minor classes and a lower selection probability for the major classes. $\Psi^0(c)$ is defined as follows.

$$\Psi^0(c) = \frac{1 - \Omega^0(c)}{\sum_{c' \in \mathcal{C}} [1 - \Omega^0(c')]} \quad (5)$$

In (5), $\sum_{c' \in \mathcal{C}} [1 - \Omega^0(c')]$ serves as a normalization factor, ensuring that the resulting values form a valid probability distribution. This results in a balanced i and j pair between the majority and minority classes.

In the first step, the pre-training loss $\mathcal{L}_{\text{pre}}(\cdot)$ for (2) is defined as follows.

$$\begin{aligned} \mathcal{L}_{\text{pre}}(\mathbf{x}_i^0, \mathbf{x}_j^0, y_i^0, y_j^0; \theta^0) = & \lambda (\mathcal{L}_{\text{cross}}(\mathbf{x}_i^0, y_i^0; \theta^0) \\ & + \mathcal{L}_{\text{cross}}(\mathbf{x}_j^0, y_j^0; \theta^0)) \\ & + (1 - \lambda) \mathcal{L}_{\text{cont}}(\mathbf{x}_i^0, \mathbf{x}_j^0, y_i^0, y_j^0; \theta_{\bar{L}}^0) \end{aligned} \quad (6)$$

It is a combination of the cross-entropy loss $\mathcal{L}_{\text{cross}}(\cdot)$ and contrastive loss $\mathcal{L}_{\text{cont}}(\cdot)$. Here, λ for $(0 \leq \lambda \leq 1)$ is a scaling

parameter that balances the two objectives. The cross-entropy loss $\mathcal{L}_{\text{cross}}(\mathbf{x}_i^0, y_i^0; \theta^0)$ is defined as follows.

$$\mathcal{L}_{\text{cross}}(\mathbf{x}_i^0, y_i^0; \theta^0) = - \sum_{c \in \mathcal{C}} p_{\mathbf{x}_i^0}(c) \log q_{\mathbf{x}_i^0}(c; \theta^0) \quad (7)$$

The objective of (7) is to minimize the gap between the probability of the true label $p_{\mathbf{x}_i^0}(c)$ and the model output $q_{\mathbf{x}_i^0}(c; \theta^0)$. Next, the contrastive loss $\mathcal{L}_{\text{cont}}(\mathbf{x}_i^0, \mathbf{x}_j^0, y_i^0, y_j^0; \theta_{\bar{L}}^0)$ in (6) is computed in the latent space, that is the output of the $\bar{L}$ -th layer for $1 < \bar{L} < L$.²

$$\mathcal{L}_{\text{cont}}(\mathbf{x}_i^0, \mathbf{x}_j^0, y_i^0, y_j^0; \theta_{\bar{L}}^0) \quad (8)$$

$$= \left[\mathbb{I}_{\{y_i^0 = y_j^0\}} (\|\theta_{\bar{L}}^0(\mathbf{x}_i^0) - \theta_{\bar{L}}^0(\mathbf{x}_j^0)\|_2) \right. \quad (9)$$

$$\left. + \mathbb{I}_{\{y_i^0 \neq y_j^0\}} (\max(0, \epsilon - \|\theta_{\bar{L}}^0(\mathbf{x}_i^0) - \theta_{\bar{L}}^0(\mathbf{x}_j^0)\|_2)) \right] \quad (10)$$

Here, (9) encourages the model to minimize the distance between same class samples, whereas (10) maintains the minimum ϵ distance to separate different class samples. To measure the distance in the latent space, we use the Euclidean distance $\|\cdot\|_2$. The margin hyperparameter $\epsilon \geq 0$ controls the separation between different class samples. The indicator function $\mathbb{I}_{\{\text{condition}\}}$ is defined as follows.

$$\mathbb{I}_{\{\text{condition}\}} = \begin{cases} 1, & \text{if the condition is true} \\ 0, & \text{otherwise} \end{cases} \quad (11)$$

The model θ^0 is updated in a supervised manner based on (2) using the proposed pre-training loss $\mathcal{L}_{\text{pre}}(\mathbf{x}_i^0, \mathbf{x}_j^0, y_i^0, y_j^0; \theta^0)$ in

²The contrastive representation layer $\bar{L}$ is chosen at the middle layer of the model, which is the end of the feature extraction and the beginning of classification. In Section VII-A, we explore the choice of $\bar{L}$ and report the corresponding performance.

(6). After this step, the model moves on to the next borderline sample refinement step.

B. Step II: Borderline Sample Refinement

In this step, we focus on the feature representations of borderline samples that are difficult to classify, aiming to pull them toward the corresponding class centroid in the latent space through the contrastive loss function (8). To identify such borderline samples, we employ the Mahalanobis distance (MD) at the $\bar{L}$ -th layer [46], [47]. MD normalizes the distance for each class to have unit variance, making it suitable for imbalanced data with varying numbers of samples and different dispersions for each class.³

Let $D_{MD}(\mathbf{x}_i^0, y_i^0, \mu_c, \Sigma_c; \theta_{\bar{L}}^0)$ be the MD of c class sample $(\mathbf{x}_i^0, y_i^0)$ at the output of $\bar{L}$ -th layer. It measures a normalized distance from the centroid as follows.

$$D_{MD}(\mathbf{x}_i^0, y_i^0, \mu_c, \Sigma_c; \theta_{\bar{L}}^0) = \sqrt{(\theta_{\bar{L}}^0(\mathbf{x}_i^0) - \mu_c)^T \Sigma_c^{-1} (\theta_{\bar{L}}^0(\mathbf{x}_i^0) - \mu_c)} \quad (12)$$

Here, $\mu_c = \mathbb{E}[\theta_{\bar{L}}^0(\mathbf{x}_i^0) \mid c = y_i^0]$ denotes the mean vector of c class samples, and $\Sigma_c = \mathbb{E}[(\theta_{\bar{L}}^0(\mathbf{x}_i^0) - \mu_c)(\theta_{\bar{L}}^0(\mathbf{x}_i^0) - \mu_c)^T \mid c = y_i^0]$ denotes the covariance matrix of class c .

Next, we determine an anchor sample $(\mathbf{x}_{a,c}^0, y_{a,c}^0)$ for each class $c \in \mathcal{C}$ and borderline samples $(\mathbf{x}_{b,c}^0, y_{b,c}^0)$ that need refinement. Note that the goal of this step is to pull the borderline samples toward the anchor sample. An anchor sample $(\mathbf{x}_{a,c}^0, y_{a,c}^0)$ is selected as the sample with the smallest MD for each class c , i.e.,

$$\mathbf{x}_{a,c}^0 = \arg \min_{\mathbf{x}_i^0 \in \mathbf{X}^0} D_{MD}(\mathbf{x}_i^0, y_i^0, \mu_c, \Sigma_c; \theta_{\bar{L}}^0). \quad (13)$$

Because there is one anchor per class, there are $|\mathcal{C}|$ anchor samples. We determine the borderline sample $(\mathbf{x}_{b,c}^0, y_{b,c}^0)$ if $D_{MD}(\mathbf{x}_{b,c}^0, y_{b,c}^0, \mu_c, \Sigma_c; \theta_{\bar{L}}^0)$ exceeds a borderline threshold ϕ_{border} , i.e., $D_{MD}(\mathbf{x}_{b,c}^0, y_{b,c}^0, \mu_c, \Sigma_c; \theta_{\bar{L}}^0) > \phi_{border}$.⁴ An illustrative example of a borderline sample is presented in Fig. 3.

In the second step, the pre-training loss $\mathcal{L}_{pre}(\cdot)$ for (2) is the same as $\mathcal{L}_{pre}(\mathbf{x}_{a,c}^0, \mathbf{x}_{b,c}^0, y_{a,c}^0, y_{b,c}^0; \theta^0)$ defined in (6), but the pair of samples $(\mathbf{x}_{a,c}^0, \mathbf{x}_{b,c}^0, y_{a,c}^0, y_{b,c}^0)$ is determined as an anchor and a borderline one for the same class. Since there is only the same class pair (i.e., $\mathbb{I}_{\{y_i^0=y_j^0\}} = 1$ and $\mathbb{I}_{\{y_i^0 \neq y_j^0\}} = 0$), the contrastive loss $\mathcal{L}_{cont}(\mathbf{x}_{a,c}^0, \mathbf{x}_{b,c}^0, y_{a,c}^0, y_{b,c}^0; \theta_{\bar{L}}^0)$ in (8) can be simplified as

³In class-imbalanced environments, different classes often exhibit varying degrees of dispersion in the latent space. Under such conditions, classical distance metrics like Euclidean distance become inappropriate, as they can yield inconsistent distance scales across classes. For example, if class A is widely spread, the distance between a borderline sample and its class center can be large, whereas in a compact class B, the same type of distance may be much smaller. To enable a unified distance based criterion for identifying borderline samples across classes, we adopt the MD, which normalizes distances within each class to have unit variance.

⁴In Fig. 4 of Section VII-A, our empirical analysis suggests that a borderline threshold of $\phi_{border} = 3$ performs best. This choice can be theoretically justified. Under the assumption of a multivariate normal distribution, a MD of 3 corresponds to the 99.7% confidence interval, a commonly accepted threshold for detecting statistical outliers.

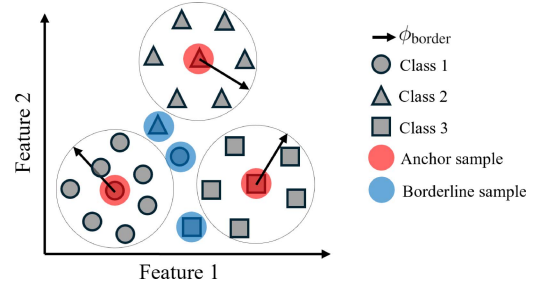


Fig. 3. An illustration of selection criteria for anchor and borderline samples. The visualization is presented in MD space with unit variance across all classes.

follows.

$$\mathcal{L}_{cont}(\mathbf{x}_{a,c}^0, \mathbf{x}_{b,c}^0, y_{a,c}^0, y_{b,c}^0; \theta_{\bar{L}}^0) = \|\theta_{\bar{L}}^0(\mathbf{x}_{a,c}^0) - \theta_{\bar{L}}^0(\mathbf{x}_{b,c}^0)\|_2 \quad (14)$$

Note that (10) is ignored because this borderline sample refinement step has no different class pairs. In (14), the contrastive loss minimizes the distance between the anchor and borderline samples. This allows refinement of the representation of borderline samples, which helps enhance classification performance.

The model θ^0 is updated in a supervised manner based on (2) using the proposed pre-training loss $\mathcal{L}_{pre}(\mathbf{x}_{a,c}^0, \mathbf{x}_{b,c}^0, y_{a,c}^0, y_{b,c}^0; \theta^0)$ in (6). Once this step is completed, the model returns to the first step. By alternating these two steps, the model progressively improves its ability to classify both majority and minority classes.

V. EFFICIENT MODEL POST-TRAINING WITH PSEUDO-LABELS

Once the model is fully trained, it is deployed in the real world. In this environment, the data label distribution often differs from that of the pre-training dataset, with a higher occurrence of minority class samples. This can lead to degraded classification accuracy, making post-training necessary. The post-training phase fine-tunes the model using only a few online unlabeled samples.

To minimize the computational burden, post-training is activated only when necessary. In this section, we address two challenges: unlabeled online data and the limited number of samples for updating the model. To tackle these issues, we introduce a pseudo-labeling method and a LoRA-based sample-efficient update method. The proposed post-training phase is shown in Fig. 2, and the pseudocode is presented in Algorithm 2.

A. Conditional Post-Training

The post-training is activated if the uncertainty level of the model prediction exceeds ϕ_{ent} and the similarity between the predicted label distribution of $\mathbf{X}^t$ and $\mathbf{X}^{t-1}$ is smaller than ϕ_{cos} . We first define the uncertainty level as the entropy $h(\mathbf{x}_i^t; \theta^t)$, which quantifies the uncertainty in the model predictions.

$$h(\mathbf{x}_i^t; \theta^t) = - \sum_{c \in \mathcal{C}} q_{\mathbf{x}_i^t}(c; \theta^t) \log q_{\mathbf{x}_i^t}(c; \theta^t) \quad (15)$$

If $h(\mathbf{x}_i^t; \theta^t)$ exceeds the threshold ϕ_{ent} , indicating a higher degree of uncertainty, the model must be fine-tuned to improve

Algorithm 2: Post-Training Phase.

Require: Model θ^t , Dataset $\mathbf{X}^t = \{\mathbf{x}_i^t\}_{i=1}^{N^t}$, $\mathbf{X}^{t-1} = \{\mathbf{x}_i^{t-1}\}_{i=1}^{N^{t-1}}$, thresholds ϕ_{ent} , ϕ_{cos}

Ensure: Updated model θ^{t+1}

- 1: Compute similarity $s(\mathbf{x}_i^t, \mathbf{x}_i^{t-1}; \theta^t)$
- 2: **for** $i = 1$ to N^t **do**
- 3: Compute entropy $h(\mathbf{x}_i^t; \theta^t)$
- 4: **if** $h(\mathbf{x}_i^t; \theta^t) \geq \phi_{\text{ent}}$ **and** $s(\mathbf{x}_i^t, \mathbf{x}_i^{t-1}; \theta^t) < \phi_{\text{cos}}$ **then**
- 5: Generate pseudo-label $\tilde{y}_i^t$ based on (17)
- 6: **if** $\tilde{y}_i^t$ is generated **then**
- 7: Compute $\mathcal{L}_{\text{post}}(\mathbf{x}_i^t, \tilde{y}_i^t; \theta^t, \{\mathbf{A}, \mathbf{B}\})$ based on (20)
- 8: Update final layer
 $\{\mathbf{A}, \mathbf{B}\} \leftarrow \{\mathbf{A}, \mathbf{B}\} - \eta \nabla \mathcal{L}_{\text{post}}(\mathbf{x}_i^t, \tilde{y}_i^t; \theta^t, \{\mathbf{A}, \mathbf{B}\})$
- 9: $\theta_L^t \leftarrow \theta_L^t + \mathbf{BA}$
- 10: **end if**
- 11: **end if**
- 12: **end for**

the prediction confidence. Next, we measure the similarity of predicted label distribution between $\mathbf{X}^t$ and $\mathbf{X}^{t-1}$. If the similarity is below the threshold ϕ_{cos} , this suggests potential shifts in the label distribution, making post-training necessary. We denote the similarity $s(\mathbf{X}^t, \mathbf{X}^{t-1}; \theta^t)$ as the cosine similarity between the outputs of the current data $\mathbf{X}^t$ and previous data $\mathbf{X}^{t-1}$.

$$s(\mathbf{X}^t, \mathbf{X}^{t-1}; \theta^t) = \frac{\sum_{c \in \mathcal{C}} (q_{\mathbf{x}^t}(c; \theta^t) \cdot q_{\mathbf{x}^{t-1}}(c; \theta^t))}{\sqrt{\sum_{c \in \mathcal{C}} (q_{\mathbf{x}^t}(c; \theta^t))^2} \cdot \sqrt{\sum_{c \in \mathcal{C}} (q_{\mathbf{x}^{t-1}}(c; \theta^t))^2}} \quad (16)$$

The model adapts when both the uncertainty and similarity conditions are satisfied, thereby ensuring that it responds effectively.

B. Pseudo-Labeling

To update the model in a supervised manner using (4), a reliable pseudo-label must be generated. An inaccurate pseudo-label can degrade the accuracy of the model. The pseudo-label $\tilde{y}_i^t$ is assigned based on both the entropy $h(\mathbf{x}_i^t; \theta^t)$ and confidence measure Δ_{MD} , ensuring that reliable pseudo-labels are generated only if certain conditions are satisfied.

$$\tilde{y}_i^t = \begin{cases} \hat{y}_i^t, & \text{if } h(\mathbf{x}_i^t; \theta^t) < \phi_{\text{pred}} \\ \arg \min_{c \in \mathcal{C}} D_{\text{MD}}(\mathbf{x}_i^t, y_i^t, \mu_c, \Sigma_c; \theta^t), & \text{if } h(\mathbf{x}_i^t; \theta^t) \geq \phi_{\text{pred}} \\ & \text{and } \Delta_{\text{MD}} \geq \phi_{\text{MD}} \\ \text{No pseudo-labeling,} & \text{otherwise} \end{cases} \quad (17)$$

If the entropy $h(\mathbf{x}_i^t; \theta^t)$ is below the threshold ϕ_{pred} , the model is confident in its prediction.⁵ Here, we directly use the prediction $\hat{y}_i^t$ as the pseudo-label, i.e., $\tilde{y}_i^t = \hat{y}_i^t$.

⁵Here, the entropy threshold ϕ_{pred} serves a different purpose from ϕ_{ent} used in deciding to perform post-training. While ϕ_{ent} determines when to initiate post-training by evaluating the uncertainty of the incoming data, ϕ_{pred} decides whether the model prediction is reliable enough to be used as a pseudo-label.

When model entropy $h(\mathbf{x}_i^t; \theta^t)$ exceeds ϕ_{pred} , we cannot rely on the model's prediction. In this case, we leverage the feature representation at $\bar{L}$ -th layer by measuring the distance to the centroid of all classes. Let $\mathcal{M}$ be a set of MD for a sample $(\mathbf{x}_i^t, y_i^t)$ for all classes.

$$\mathcal{M} = \{D_{\text{MD}}(\mathbf{x}_i^t, y_i^t, \mu_c, \Sigma_c; \theta^t) | c \in \mathcal{C}\} \quad (18)$$

The set $\mathcal{M}$ contains $|\mathcal{C}|$ elements. A straightforward approach is to assign a pseudo-label to the class with the smallest distance, i.e., $\tilde{y}_i^t = \arg \min_{c \in \mathcal{C}} D_{\text{MD}}(\mathbf{x}_i^t, \mu_c, \Sigma_c; \theta^t)$. To ensure a more reliable pseudo-label, we further assess the confidence measure Δ_{MD} , which represents the distance gap between the first and second closest class centroids.

$$\Delta_{\text{MD}} = \min(\mathcal{M} \setminus \{\min(\mathcal{M})\}) - \min(\mathcal{M}) \quad (19)$$

Here, $\min(\cdot)$ returns the element with the minimum value in the set and $\setminus$ denotes the set difference. We employ the smallest distance class as a pseudo-label only if the confidence measure Δ_{MD} exceeds ϕ_{MD} . If any of these conditions are not met, we avoid pseudo-labeling and exclude the sample from the post-training process.

C. LoRA-Based Sample-Efficient Post-Training

After determining the pseudo-labels, we finally update the model. Because of a very limited number of samples for this phase, i.e., $N^t \ll N^0$, we update only the final layer of the model [48], [49], [50]. In addition, we employ LoRA to reduce the number of trainable parameters.

The post-training loss function $\mathcal{L}_{\text{post}}(\cdot)$ in (4) is defined as a cross-entropy loss with a pseudo-label.

$$\begin{aligned} \mathcal{L}_{\text{post}}(\mathbf{x}_i^t, \tilde{y}_i^t; \theta^t, \{\mathbf{A}, \mathbf{B}\}) \\ = - \sum_{c \in \mathcal{C}} \tilde{p}_{\mathbf{x}_i^t}(c) \left(\log q_{\theta_{L-1}^t}(\mathbf{x}_i^t)(c; \theta_L^t + \mathbf{BA}) \right) \end{aligned} \quad (20)$$

The matrix $\mathbf{B} \in \mathbb{R}^{d_{L-1} \times r}$ and matrix $\mathbf{A} \in \mathbb{R}^{r \times C}$ are low-rank matrices that significantly reduce the post-training complexity compared to updating the entire layer. At the beginning of the post-training phase, $\mathbf{B}$ is set to all zero matrices, whereas $\mathbf{A}$ is initialized using Gaussian random values. The product $\mathbf{BA}$ starts at zero, ensuring that there is no initial impact on the model. As the post-training progresses, $\mathbf{B}$ and $\mathbf{A}$ are updated, enhancing the classification accuracy and efficient post-training.

Finally, the model is updated as follows.

$$\{\mathbf{A}, \mathbf{B}\} \leftarrow \{\mathbf{A}, \mathbf{B}\} - \eta \nabla \mathcal{L}_{\text{post}}(\mathbf{x}_i^t, \tilde{y}_i^t; \theta^t, \{\mathbf{A}, \mathbf{B}\}) \quad (21)$$

$$\theta_L^t \leftarrow \theta_L^t + \mathbf{BA} \quad (22)$$

Here, the term $\mathbf{BA}$ is continuously merged into the base parameter θ_L^t after each update step, rather than being applied only once after training. This continuous write-back follows the standard LoRA update mechanism [45], [51], [52], ensuring that the effective weight $\theta_L^t + \mathbf{BA}$ is directly optimized during post-training.

VI. SIMULATION SETUPS

In this section, the simulation environment used to evaluate the proposed system is described. Our experiments are conducted

on multiple datasets, simulating online label shift scenarios to test the adaptability and robustness of our approach compared to several baseline algorithms.

A. Datasets

We utilized three well-known network intrusion classification datasets that are commonly used to evaluate intrusion detection systems. These datasets provide a comprehensive range of normal traffic and attack classes, making them suitable for testing the dynamic post-training mechanisms.

- *NSL-KDD [53]*: An improved version of the KDD Cup 99 dataset [54], NSL-KDD addresses issues such as redundant records and includes 41 features across approximately 125,973 records. It is divided into five classes: Normal, DoS, Probe, R2L, and U2R.
- *UNSW-NB15 [55]*: A hybrid dataset combining real and synthetic network traffic. It contains 49 features for each connection and a total of 2,540,044 records, covering 10 different classes, including various types of attacks like exploits, fuzzers, and worms.
- *ToN-IoT [56]*: Designed for cybersecurity research in IoT networks, ToN-IoT comprises 46 features and over 22,339,021 records, spanning nine classes such as normal traffic and several types of attacks like DoS, DDoS, and scanning.

The dataset was partitioned into pre-training and post-training sets with a fixed ratio of 8:2. Within the pre-training set, data were further divided into training and validation subsets at a ratio of 3:1. All datasets were preprocessed using z-score normalization, where each feature was standardized based on the mean and standard deviation calculated from the pre-training data. At each post-training time step t ($1 \leq t \leq 100$), 100 distinct post-training sets are independently drawn from post-training sets according to the label distribution $\Omega^t(c)$. The performance of the model is evaluated using post-training sets at every time step t , and we report the average performance across all steps. This evaluation setting is aligned with standard practices in the label distribution shift literature [35], [36], [57].

B. Online Label Shift

As defined in (3), the online label distribution shift is determined by the initial distribution $\omega_1(c)$, the final distribution $\omega_2(c)$, and a time-dependent parameter α^t , which controls the distribution shift over time. Here, α^t is typically configured in the following four ways under online label distribution shift scenarios [35], [36], [57].

- *Pre-trained Model Performance (Pre)*: The accuracy evaluated, before any label distribution shift is applied. This baseline assesses the model's initial classification performance under the same class distribution as the pre-training data.
- *Linear Shift (Lin)*: $\alpha^t = \frac{t}{T}$ provides a gradual, linear transition from $\omega_1(c)$ to $\omega_2(c)$, ensuring a smooth evolution in the label distribution.
- *Square Shift (Squ)*: α^t toggles between 0 and 1 every $\sqrt{T}/2$ iterations. This creates abrupt, periodic changes in the class distribution.

- *Sine Shift (Sin)*: $\alpha^t = \sin(\frac{t\pi}{\sqrt{T}})$ introduces cyclical variations, capturing periodic shifts in the distribution.
- *Bernoulli Shift (Ber)*: α^t retains its previous value α^{t-1} with probability $\frac{1}{\sqrt{T}}$, or flips to $1 - \alpha^{t-1}$. This setup captures stochastic variations, ensuring class prior changes scale with $\sqrt{T}$.

The visualizations of α^t across scenarios are presented in Appendix C, available online.

We categorize the online label distribution settings into Dynamic Setting I and Dynamic Setting II, based on how the initial distribution $\omega_1(c)$ and the final distribution $\omega_2(c)$ are defined. In Dynamic Setting I, $\omega_1(c)$ is set to a uniform class distribution, whereas $\omega_2(c)$ is set to a Dirac delta distribution, which values zero, except for the most minor class of the pre-training phase. This setting allows for an extreme label shift scenario, where the class with the lowest proportion in the pre-training phase becomes the most prevalent during the post-training phase [35], [36], [57]. Although Dynamic Setting I is the most popular setting in literature, it only partially mirrors real-world conditions because configuring the initial distribution as a uniform distribution fails to account for class imbalance. As the final distribution converges to a single class, the model achieves the lowest loss by consistently predicting this class, making the task easier than in settings with diverse class inclusion.

To consider a more realistic setting, we propose a novel online shift setting, Dynamic Setting II. In this setting, we configure the initial setting $\omega_1(c)$ as a non-uniform distribution to account for class imbalance and establish a final setting $\omega_2(c)$ to include diverse classes. The setup for $\omega_1(c)$ includes normal traffic samples selected to exceed $N^t/|C|$, ensuring that normal data are the majority. The remaining attack classes are then randomly selected with counts below $N^t/|C|$. For $\omega_2(c)$, normal traffic also maintains the majority, but the least frequent attack class from pre-training is set as the most common attack class, with counts between those of normal traffic and other attack classes. The other attack classes are selected randomly to remain below $N^t/|C|$. Unlike in Dynamic Setting I, the specific attack classes included in $\omega_1(c)$ and $\omega_2(c)$ vary with each random seed, allowing for a more realistic consideration of the label distribution environment. For each dataset, the initial and final distributions for Setting I and Setting II can be found in Appendix C, available online, along with numerical values.

While the two settings differ in how the initial and final label distributions are defined, the degree of imbalance in each setting can be quantitatively characterized by the imbalance factor (IF), which is widely used in the literature to measure class imbalance. The imbalance factor is defined as $IF = \frac{N_{\max}}{N_{\min}}$, where $N_{\max}$ and $N_{\min}$ denote the maximum and minimum number of samples among all classes, respectively. A larger IF indicates a more severe skew in the class distribution.

C. Baseline Algorithms

To benchmark the performance of the proposed method, we compared it with several baseline models tailored for pre-training and post-training. All models use the same architecture as ours, ensuring a fair comparison across different training

mechanisms. The detailed hyperparameter settings are provided in Appendix A, available online.

1) Pre-Training Baseline Algorithms:

- *Base*: This method trains a classifier solely with cross-entropy loss. It serves as the simplest baseline without any modifications to the data or learning process. Unlike our method, it does not incorporate any techniques for handling class imbalance.
- *Ours*: The proposed method integrates cross-entropy loss, contrastive loss, class imbalance handling, and borderline sample refinement to jointly training classification and representation.
- *Random Oversampling* [27]: A simple approach that randomly duplicates samples from minority classes to balance the dataset. The model is trained solely with cross-entropy loss on the resampled dataset. Unlike our method, it does not generate new representations but merely increases the presence of existing minority class instances, potentially leading to overfitting.
- *Synthetic Minority Oversampling Technique (SMOTE)* [18]: This method generates synthetic samples to balance the dataset before training, using interpolation between existing instances. The model is then trained solely with cross-entropy loss on the augmented dataset. However, unlike our method, SMOTE does not refine class boundaries dynamically or leverage contrastive learning for improved feature representation.
- *Borderline-SMOTE* [29]: An advanced version of SMOTE that focuses on generating synthetic samples near the decision boundary of minority classes. The model trains solely with cross-entropy loss after augmentation. While it improves class separability compared to standard SMOTE, it struggles with the joint optimization of classification and representation learning that our method integrates.
- *Focal Loss* [30]: A loss function designed to address class imbalance by assigning higher weights to hard-to-classify instances. It helps mitigate the dominance of majority classes by reducing the contribution of well-classified samples. However, unlike our method, it does not incorporate contrastive learning or explicit sample refinement mechanisms to enhance representation learning.

2) Post-Training Baseline Algorithms:

- *Base (no post-training)*: This model uses the pre-trained model without post-training. It serves as a lower bound to highlight the improvements achieved through post-training strategies.
- *Ours*: This is the proposed method that integrates a confidence-based mechanism with a LoRA. It dynamically generates pseudo-labels based on the confidence of the model and selectively updates the model to optimize both accuracy and efficiency.
- *Follow the History (FTH)* [35]: This algorithm averages label distributions across the entire history, assuming that past data can inform future patterns. It does not use confidence-based pseudo-labeling, which may limit its responsiveness to abrupt changes.
- *Follow the Fixed Window History (FTFWH)* [35]: FTFWH adapts using a fixed window of recent data, focusing on the

most current observations. Although this method responds to rapid label shifts, it struggles with the confidence-based selective post-training employed in our approach, which may reduce its effectiveness in complex scenarios.

- *Unbiased Online Gradient Descent (UOGD)* [36]: An extension of OGD, UOGD uses an unbiased risk estimator to adapt continuously without labeled data. However, it does not apply confidence measures for pseudo-label selection, which might hinder its performance when faced with rapid distribution changes.
- *Adapting to Label Shift (ATLAS)* [36]: ATLAS maintains an ensemble of models, each tuned with different learning rates to adjust to label shifts. It dynamically selects the most suitable model based on current conditions. It does not leverage a confidence-based post-training mechanism or low-rank updates, which may affect its adaptability and efficiency under dynamic shifts.

All simulation results are averaged over ten random seeds with one standard deviation. Accuracy and F1-score are used as the evaluation metric. For performance comparison, the improvement of the proposed method over the best-performing baseline is expressed as a relative improvement rate, as shown below.

$$\text{Improvement (\%)} = \frac{\text{Metric}_{\text{proposed}} - \text{Metric}_{\text{best baseline}}}{\text{Metric}_{\text{best baseline}}} \times 100 \quad (23)$$

VII. SIMULATION RESULTS FOR PRE-TRAINING

In this section, we present the simulation results of the proposed system. We evaluate the proposed pre-training method through three analyses. First, we conduct a parameter sensitivity study to examine the impact of different hyperparameter settings. Second, we perform an ablation study on Step I and Step II of the proposed pre-training method to assess their individual contributions. Finally, we compare our approach with baseline pre-training algorithms to validate its effectiveness. The convergence behavior of the pre-training phase is presented in the loss curves shown in Appendix B, available online.

A. Parameter Sensitivity Analyses

To examine the engineering trade-offs underlying the proposed framework, we perform a parameter sensitivity study by varying key design parameters. All evaluations are performed on a validation dataset with a uniform label distribution [35], [36], [57], which provides a consistent basis for isolating the effects of individual design choices. The results are summarized in Fig. 4.

$\bar{L}$ determines the network layer at which the contrastive constraint is enforced. Enforcing contrastive loss at early layers preserves general feature characteristics but provides limited class discrimination, whereas applying it at very late layers yields diminishing returns, as the representations become increasingly specialized for classification. As shown in Fig. 4(a), performance improves as $\bar{L}$ moves toward intermediate layers, where the contrastive objective can effectively shape the feature space without interfering with class-specific decision boundaries. Beyond this point, further increasing $\bar{L}$ offers no additional benefit, indicating that intermediate feature layers provide the

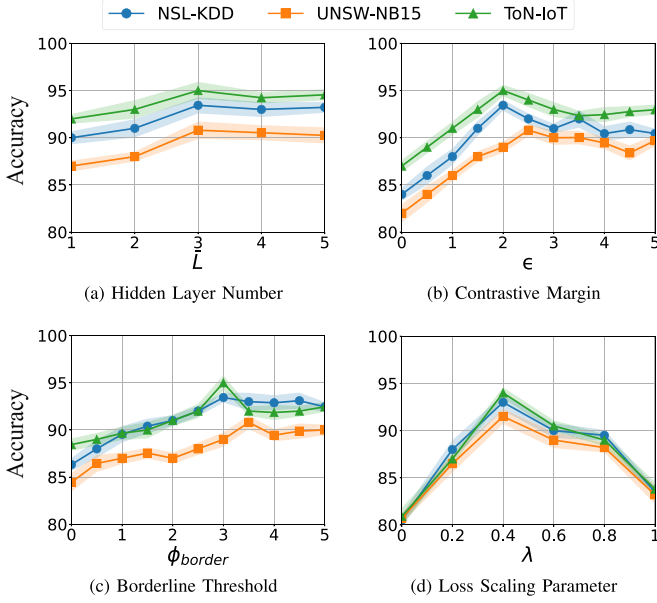


Fig. 4. Parameter sensitivity analysis results. The subfigures illustrate the impact of varying key parameters on the performance of our proposed pre-training method: (a) the constant $\bar{L}$ indicates the hidden layer number, (b) the margin parameter ϵ for contrastive learning, (c) the threshold ϕ_{border} used for selecting borderline samples, and (d) scaling parameter λ used to balance two objectives.

most effective balance between representation robustness and discriminability.

The contrastive margin parameter ϵ controls how strongly samples from different classes are separated in the learned feature space. An insufficient margin fails to enforce meaningful separation between minority classes, limiting discriminability, whereas an excessively large margin can distort the feature geometry and destabilize representation learning. As shown in Fig. 4(b), increasing ϵ initially improves performance by promoting clearer class separation. Beyond a moderate range, however, further enlarging the margin yields diminishing returns, indicating that once adequate separation is achieved, additional margin constraints no longer contribute to more informative representations.

The threshold ϕ_{border} is employed during pre-training to identify borderline samples, allowing the model to focus on instances that are difficult to classify. As illustrated in Fig. 4(c), a threshold of $\phi_{\text{border}} \approx 3$ yields the best performance across three datasets. Theoretically, an MD of 3 corresponds to the 99.7% confidence interval under the assumption of a multivariate normal distribution. Therefore, our empirical findings are well supported by a widely accepted statistical criterion for detecting outliers, suggesting an initial setting of $\phi_{\text{border}} = 3$, followed by tuning based on empirical results.

The scaling parameter λ balances the two objectives in the pre-training loss. As shown in Fig. 4(d), the accuracy increases sharply as λ rises to around 0.4, where the contributions of the cross-entropy and contrastive losses are better balanced. Beyond this point, the performance gradually decreases toward $\lambda = 1$, indicating that assigning excessive weight to either objective does not further improve feature discriminability. A moderate

λ therefore provides a well-balanced contribution from the two losses.

B. Ablation Studies

In the pre-training phase, the proposed system performs two key steps: imbalance-aware training (Step I), which addresses data imbalance issues, and borderline sample refinement (Step II), which helps classify samples near decision boundaries within the latent space. The ablation study analyzes how each step affects the initial performance after pre-training and how this initial performance influences the post-training results under the same conditions. The results are summarized in Table IV. The cases are defined as follows:

- *Case 1*: Neither Step I nor Step II is performed. This corresponds to a vanilla contrastive learning baseline where sample pairs are selected randomly without considering class imbalance or borderline sample refinement.
- *Case 2*: Only Step I is performed, where contrastive pairs are selected with higher probability for minority classes to address class imbalance, but no borderline sample refinement is done.
- *Ours*: Both Step I and Step II are performed. The model applies imbalance-aware training and further refines borderline samples near class decision boundaries.

When comparing the initial performance of Case 1 and Case 2, we observe that including Step I leads to improved initial performance, demonstrating that imbalance-aware training effectively addresses data imbalance issues. Furthermore, a comparison between Case 2 and Ours reveals that Ours achieves superior initial performance. This suggests that while imbalance-aware training mitigates data imbalance, samples near decision boundaries continue to pose classification challenges. By applying borderline sample refinement in Step II, the classification accuracy improves for these challenging samples, leading to a better initial performance. An analysis of the impact of initial performance on the post-training phase shows that post-training results are achieved when initial performance is higher.

C. Performance Comparisons

Table V presents a comparison of various methods across multiple datasets. Our proposed method consistently outperforms all baselines, achieving an average improvement of 6.0% in accuracy and F1-score over the best-performing comparison algorithm across all datasets. Traditional resampling techniques, such as SMOTE [18] and Borderline-SMOTE [29], improve performance by increasing the presence of minority class samples. However, these methods rely on synthetic data interpolation, limiting their ability to capture the full complexity of real-world variations. Loss reweighting approaches, including Focal Loss [30], further enhance performance by focusing on hard-to-classify samples. While effective, these methods depend on predefined weighting strategies, which may not generalize well across different data distributions.

This result suggests that our approach effectively captures intricate class relationships, enabling superior generalization

TABLE IV
PRE-TRAINED MODEL PERFORMANCE AND ITS ABLATION STUDIES

Accuracy [%]																	
Case	Step I	Step II	NSL-KDD [53]					UNSW-NB15 [55]					ToN-IoT [56]				
			Pre	Lin	Squ	Sin	Ber	Pre	Lin	Squ	Sin	Ber	Pre	Lin	Squ	Sin	Ber
Case 1	✗	✗	87.2 ±0.11	84.9 ±0.13	84.3 ±0.12	84.5 ±0.14	83.7 ±0.15	84.7 ±0.16	82.7 ±0.13	82.1 ±0.14	81.9 ±0.16	81.5 ±0.13	88.3 ±0.12	85.3 ±0.14	85.2 ±0.17	85.0 ±0.12	84.4 ±0.15
Case 2	✓	✗	90.1 ±0.18	86.8 ±0.15	86.1 ±0.18	85.7 ±0.16	85.1 ±0.13	87.4 ±0.12	84.6 ±0.14	84.2 ±0.16	83.6 ±0.15	83.1 ±0.14	90.8 ±0.13	86.3 ±0.12	85.7 ±0.15	85.2 ±0.16	84.6 ±0.13
Ours	✓	✓	93.4 ±0.13	90.3 ±0.14	89.8 ±0.17	89.4 ±0.15	89.1 ±0.12	90.8 ±0.13	88.1 ±0.16	87.8 ±0.14	87.5 ±0.17	87.2 ±0.15	94.9 ±0.14	92.6 ±0.15	92.2 ±0.13	92.4 ±0.18	92.3 ±0.14

TABLE V
PERFORMANCE COMPARISON OF PRE-TRAINED MODEL ACROSS MULTIPLE DATASETS (ACCURACY [%] AND F1-SCORE [%])

Data		NSL-KDD [53]		UNSW-NB15 [55]		ToN-IoT [56]	
Metric		Accuracy	F1-score	Accuracy	F1-score	Accuracy	F1-score
Method	Base	82.7±0.09	81.5±0.13	80.6±0.13	79.1±0.14	79.5±0.12	78.0±0.13
	Random Oversampling [27]	83.8±0.11	82.1±0.12	82.3±0.10	80.9±0.13	80.7±0.13	79.5±0.14
	SMOTE [18]	85.4±0.12	84.1±0.14	83.6±0.11	82.0±0.12	82.1±0.14	80.8±0.13
	Borderline-SMOTE [29]	87.1±0.10	85.9±0.13	86.0±0.09	84.7±0.12	84.7±0.12	83.1±0.14
	Focal Loss [30]	88.0±0.11	86.9±0.12	86.2±0.10	85.0±0.13	85.0±0.11	83.9±0.12
	Ours	93.4±0.13	91.9±0.14	90.8±0.13	89.2±0.12	94.9±0.14	92.5±0.13

in diverse and dynamic environments. Unlike augmentation-based methods, our system learns more discriminative feature representations, leading to significant improvements in network intrusion detection.

These results highlight the limitations of traditional techniques and the advantages of the proposed method, demonstrating its robustness in handling imbalanced and complex data scenarios.

VIII. SIMULATION RESULTS FOR POST-TRAINING

We evaluate the proposed post-training method through a series of analyses. First, we conduct a parameter sensitivity study to examine the impact of different hyperparameter settings. Second, we perform an ablation study on the three key components of post-training—reliable pseudo-labeling, LoRA, and conditional post-training—to assess their individual contributions. Additionally, we analyze the computational complexity of LoRA to evaluate its efficiency in resource-constrained environments. We further conduct a robustness evaluation of the overall system and pseudo-labeling method to assess stability under varying data conditions. Finally, we compare the performance of our approach with baseline post-training algorithms to validate its effectiveness.

A. Parameter Sensitivity Analyses

To regulate model adaptation during post-training, the proposed framework relies on confidence-based pseudo-label selection to prevent performance degradation caused by unreliable supervision. In particular, pseudo-labels are generated only when sufficient confidence is established at both the prediction

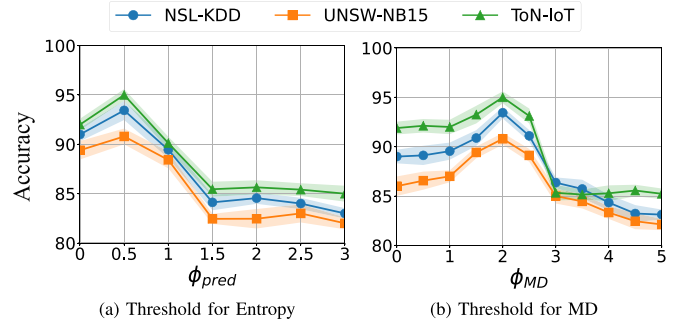


Fig. 5. Parameter sensitivity analysis results of proposed post-training method: (a) the threshold ϕ_{pred} for determining entropy confidence and (b) the threshold ϕ_{MD} for determining MD confidence.

level and the feature level. This design introduces an inherent trade-off between pseudo-label reliability and adaptation coverage, which is controlled through the entropy threshold ϕ_{pred} and the Mahalanobis distance threshold ϕ_{MD} . We examine the impact of these parameters through a sensitivity analysis, as summarized in Fig. 5.

The entropy threshold ϕ_{pred} controls how strictly prediction uncertainty is filtered during pseudo-labeling. Prediction entropy reflects the concentration of the softmax output, where low entropy indicates a dominant class probability and high entropy indicates ambiguous predictions. When ϕ_{pred} is set to a small value, pseudo-labels are assigned only to highly confident predictions, which preserves label reliability but limits adaptation by excluding most samples. As ϕ_{pred} increases, predictions with less concentrated softmax outputs are progressively included, expanding adaptation coverage. However, overly permissive

TABLE VI
POST-TRAINING PERFORMANCE, PROCESS TIME, AND ITS ABLATION STUDIES

Case	(a)	(b)	(c)	Measure	NSL-KDD [53]				UNSW-NB15 [55]				ToN-IoT [56]			
					Lin	Squ	Sin	Ber	Lin	Squ	Sin	Ber	Lin	Squ	Sin	Ber
Case 3	✗	✓	✓	Accuracy [%]	81.4 ±0.18	80.8 ±0.16	80.3 ±0.17	79.5 ±0.15	79.3 ±0.17	78.7 ±0.18	78.2 ±0.15	77.5 ±0.17	80.1 ±0.16	79.5 ±0.14	78.9 ±0.15	78.1 ±0.18
				Wall time [sec]	36.2 ±3.9	73.5 ±4.3	38.6 ±3.8	60.4 ±5.0	37.8 ±4.7	78.2 ±3.9	32.5 ±4.1	60.7 ±4.6	38.9 ±3.7	75.6 ±3.4	34.8 ±3.9	65.5 ±4.5
Case 4	✓	✗	✓	Accuracy [%]	87.5 ±0.16	87.0 ±0.17	86.6 ±0.15	86.3 ±0.13	84.8 ±0.18	84.5 ±0.14	83.9 ±0.17	83.5 ±0.16	88.9 ±0.14	87.6 ±0.15	88.0 ±0.16	87.9 ±0.15
				Wall time [sec]	65.3 ±4.2	77.4 ±4.7	66.8 ±3.9	61.5 ±3.8	66.1 ±4.5	78.1 ±4.6	68.2 ±4.2	63.2 ±3.9	64.9 ±3.6	77.9 ±4.3	67.5 ±3.8	62.9 ±4.7
Case 5	✓	✓	✗	Accuracy [%]	90.1 ±0.15	90.0 ±0.15	89.2 ±0.16	89.3 ±0.13	88.2 ±0.15	88.0 ±0.14	87.6 ±0.16	87.3 ±0.14	92.5 ±0.13	92.3 ±0.13	92.3 ±0.17	92.4 ±0.14
				Wall time [sec]	91.4 ±4.7	90.5 ±3.9	93.2 ±4.6	91.9 ±3.8	90.5 ±4.3	93.7 ±4.1	92.4 ±3.7	92.5 ±4.6	92.1 ±4.2	97.3 ±4.1	94.3 ±3.5	92.3 ±4.2
Ours	✓	✓	✓	Accuracy [%]	90.3 ±0.14	89.8 ±0.17	89.4 ±0.15	89.1 ±0.12	88.1 ±0.16	87.8 ±0.14	87.5 ±0.17	87.2 ±0.15	92.6 ±0.15	92.2 ±0.13	92.4 ±0.18	92.3 ±0.14
				Wall time [sec]	29.4 ±3.8	71.2 ±4.6	28.7 ±4.2	60.4 ±4.1	30.1 ±3.6	77.5 ±3.9	25.6 ±3.7	60.3 ±4.2	28.2 ±3.9	73.8 ±4.5	29.5 ±3.8	60.9 ±4.3

thresholds admit predictions with ambiguous class probabilities, increasing the likelihood of incorrect pseudo-labels and degrading post-training performance. As shown in Fig. 5(a), the observed performance trend reflects this trade-off between prediction-level reliability and adaptation coverage.

The Mahalanobis distance threshold ϕ_{MD} provides a feature-level confidence criterion for samples that are not directly pseudo-labeled based on prediction entropy. For a given sample, the confidence measure Δ_{MD} is defined as the difference between the smallest and second smallest Mahalanobis distances to class-wise feature distributions. A small Δ_{MD} indicates that the sample lies close to multiple class clusters and is therefore difficult to assign reliably, whereas a large value suggests clear association with a single class. A smaller threshold admits ambiguous samples and increases adaptation coverage, while a larger threshold restricts pseudo-labeling to samples with clear class separation, improving reliability at the cost of reduced adaptability. As shown in Fig. 5(b), post-training performance is shaped by how this balance between feature-level reliability and adaptation coverage is resolved.

B. Ablation Studies

To analyze the impact of each component within the post-training phase on the model performance and computational time, we conduct an ablation study on each component individually. The results of the ablation study are presented in Table VI.

- Reliable pseudo-labeling* addresses the issue of inaccurate pseudo-labels degrading model accuracy using the entropy $h(\mathbf{x}_i^t; \theta^t)$ and the confidence measure Δ_{MD} .
- LoRA* reduces the number of parameters to be updated, enabling sample-efficient post-training with limited samples.
- Conditional post-training* determines whether to proceed with post-training based on the current prediction

confidence and data similarity with the previous timestep, ensuring efficient post-training.

Without reliable pseudo-labeling in Case 3, there is the most significant performance drop and a slight increase in the processing time. This indicates that inaccurate labeling significantly affects performance. For computational efficiency, we observed that when reliable pseudo-labeling was not employed, the model exhibited a decrease in prediction confidence owing to inaccurate labeling. This drop in confidence triggers more frequent conditions for conditional post-training, leading to an increase in the number of post-training events. The absence of reliable pseudo-labeling resulted in a higher frequency of conditional post-training triggers, thereby necessitating additional training and longer processing times.

Excluding LoRA in Case 4 results in both performance degradation and an increase in computational cost. Since post-training is performed on a limited number of samples, updating all parameters in the target layers becomes ineffective due to insufficient sample size. These results confirm that LoRA enables computationally efficient post-training by reducing the number of updated parameters without sacrificing model effectiveness.

When examining the results of Case 5 and Ours, both approaches exhibit similar performance levels. However, Case 5 requires a significantly longer processing time. This indicates that performing post-training at every timestep is inefficient, while conditional post-training allows for efficient adjustments by triggering post-training only when necessary, achieving performance comparable to adapting at all timesteps.

C. Analyses of Low-Rank Adaptation

To analyze the impact of LoRA on post-training, we compare accuracy and wall time between cases with LoRA and non-LoRA, as illustrated in Fig. 6. The figure shows that when the number of post-training samples is small, LoRA not only outperforms the non-LoRA method in accuracy but also exhibits

TABLE VII
PERFORMANCE COMPARISON OF POST-TRAINING PERFORMANCE IN DYNAMIC SETTING I

Method	NSL-KDD [53]				UNSW-NB15 [55]				ToN-IoT [56]			
	Lin	Squ	Sin	Ber	Lin	Squ	Sin	Ber	Lin	Squ	Sin	Ber
Accuracy [%]												
Base	75.2 $\pm$ 0.16	74.6 $\pm$ 0.14	74.1 $\pm$ 0.16	73.6 $\pm$ 0.14	72.9 $\pm$ 0.13	72.3 $\pm$ 0.15	71.8 $\pm$ 0.16	71.2 $\pm$ 0.14	76.7 $\pm$ 0.16	76.1 $\pm$ 0.15	75.5 $\pm$ 0.14	74.9 $\pm$ 0.16
FTH [35]	82.1 $\pm$ 0.13	81.5 $\pm$ 0.16	81.0 $\pm$ 0.14	80.5 $\pm$ 0.15	79.6 $\pm$ 0.16	79.0 $\pm$ 0.13	78.4 $\pm$ 0.16	78.0 $\pm$ 0.14	83.4 $\pm$ 0.14	82.8 $\pm$ 0.12	82.2 $\pm$ 0.13	81.7 $\pm$ 0.17
FTFWH [35]	82.5 $\pm$ 0.17	81.9 $\pm$ 0.12	81.4 $\pm$ 0.14	80.8 $\pm$ 0.13	79.5 $\pm$ 0.13	78.9 $\pm$ 0.17	78.3 $\pm$ 0.16	77.8 $\pm$ 0.12	84.6 $\pm$ 0.15	83.9 $\pm$ 0.16	83.3 $\pm$ 0.13	82.8 $\pm$ 0.15
UOGD [36]	85.3 $\pm$ 0.17	80.6 $\pm$ 0.14	80.1 $\pm$ 0.13	79.5 $\pm$ 0.15	82.2 $\pm$ 0.14	77.3 $\pm$ 0.16	76.8 $\pm$ 0.12	76.1 $\pm$ 0.17	85.8 $\pm$ 0.15	81.4 $\pm$ 0.13	80.9 $\pm$ 0.15	80.3 $\pm$ 0.17
ATLAS [36]	85.1 $\pm$ 0.14	84.6 $\pm$ 0.17	84.0 $\pm$ 0.15	83.5 $\pm$ 0.12	82.1 $\pm$ 0.16	81.7 $\pm$ 0.14	81.1 $\pm$ 0.16	80.5 $\pm$ 0.14	85.4 $\pm$ 0.14	84.8 $\pm$ 0.17	84.2 $\pm$ 0.12	83.6 $\pm$ 0.14
Ours	90.3$\pm$0.14	89.8$\pm$0.17	89.4$\pm$0.15	89.1$\pm$0.12	88.1$\pm$0.16	87.8$\pm$0.14	87.5$\pm$0.17	87.2$\pm$0.15	92.6$\pm$0.15	92.2$\pm$0.13	92.4$\pm$0.18	92.3$\pm$0.14
Improvement [%]	6.9	7.1	7.5	7.8	8.9	7.9	8.7	7.8	7.5	7.1	8.2	8.7
F1-score [%]												
Base	75.6 $\pm$ 0.17	75.0 $\pm$ 0.15	74.5 $\pm$ 0.16	74.0 $\pm$ 0.14	73.3 $\pm$ 0.13	72.7 $\pm$ 0.16	72.2 $\pm$ 0.17	71.7 $\pm$ 0.15	77.1 $\pm$ 0.16	76.5 $\pm$ 0.15	75.9 $\pm$ 0.14	75.3 $\pm$ 0.17
FTH [35]	82.6 $\pm$ 0.14	82.0 $\pm$ 0.17	81.5 $\pm$ 0.15	81.0 $\pm$ 0.16	80.2 $\pm$ 0.17	79.6 $\pm$ 0.14	79.0 $\pm$ 0.17	78.6 $\pm$ 0.15	83.9 $\pm$ 0.15	83.3 $\pm$ 0.13	82.7 $\pm$ 0.14	82.2 $\pm$ 0.18
FTFWH [35]	83.0 $\pm$ 0.18	82.4 $\pm$ 0.13	81.9 $\pm$ 0.15	81.3 $\pm$ 0.14	80.0 $\pm$ 0.14	79.4 $\pm$ 0.18	78.8 $\pm$ 0.17	78.3 $\pm$ 0.13	85.1 $\pm$ 0.16	84.4 $\pm$ 0.17	83.8 $\pm$ 0.14	83.3 $\pm$ 0.16
UOGD [36]	85.7 $\pm$ 0.18	81.0 $\pm$ 0.15	80.5 $\pm$ 0.14	79.9 $\pm$ 0.16	82.7 $\pm$ 0.15	77.8 $\pm$ 0.17	77.3 $\pm$ 0.13	76.6 $\pm$ 0.18	86.2 $\pm$ 0.16	81.8 $\pm$ 0.14	81.3 $\pm$ 0.16	80.7 $\pm$ 0.18
ATLAS [36]	85.6 $\pm$ 0.15	85.1 $\pm$ 0.18	84.5 $\pm$ 0.16	84.0 $\pm$ 0.13	82.6 $\pm$ 0.17	82.2 $\pm$ 0.15	81.6 $\pm$ 0.17	81.0 $\pm$ 0.15	85.9 $\pm$ 0.15	85.3 $\pm$ 0.18	84.7 $\pm$ 0.13	84.1 $\pm$ 0.15
Ours	90.7$\pm$0.15	90.2$\pm$0.18	89.8$\pm$0.16	89.5$\pm$0.13	88.5$\pm$0.17	88.2$\pm$0.15	87.9$\pm$0.18	87.6$\pm$0.16	93.0$\pm$0.16	92.6$\pm$0.14	92.8$\pm$0.19	92.7$\pm$0.15
Improvement [%]	6.8	6.7	7.0	7.9	8.8	7.8	8.1	7.8	7.1	7.2	7.4	8.4

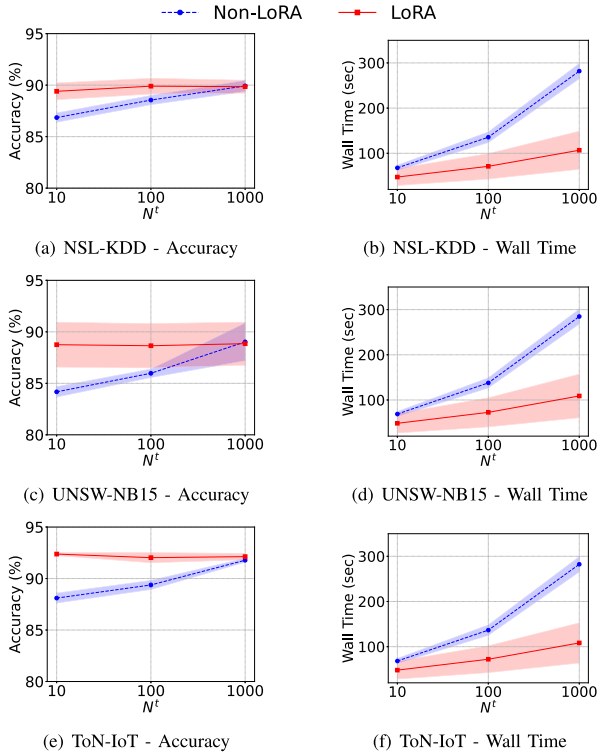


Fig. 6. Performance analysis across different datasets: (a), (c), (e) accuracy and (b), (d), (f) wall time for three datasets under different number of post-training samples N^t .

TABLE VIII
THE NUMBER OF UPDATED PARAMETERS AND COMPUTATIONAL COMPLEXITY

Dataset	Full Rank	Low Rank	Reduction [%]
NSL-KDD [53]	135	96	28.89
UNSW-NB15 [55]	320	126	60.63
ToN-IoT [56]	270	117	56.67
Computational Complexity	$\mathcal{O}(C \cdot d_{L-1})$	$\mathcal{O}(r \cdot (C + d_{L-1}))$	

greater robustness. As the number of post-training samples increases, the accuracy gap between LoRA and non-LoRA narrows. In the $N^t = 1000$ case, they achieve comparable accuracy. However, LoRA requires significantly less adaptation time, indicating a notable computational efficiency when scaling to larger sample sizes.

We further investigate the computational complexity implications of using LoRA for updating only the final layer of the model, as summarized in Table VIII. When non-LoRA is applied, the number of parameters that require updates during post-training increases substantially, leading to higher computational demands. Specifically, updating the full weight matrix incurs a computational complexity of $\mathcal{O}(C \cdot d_{L-1})$ per step, where C denotes the number of classes and d_{L-1} is the dimension of the hidden layer preceding the final layer. In contrast, LoRA decomposes the update into two low-rank matrices, $\mathbf{A} \in \mathbb{R}^{r \times C}$ and $\mathbf{B} \in \mathbb{R}^{d_{L-1} \times r}$, reducing the complexity to $\mathcal{O}(r \cdot (C + d_{L-1}))$. Since the rank r satisfies $r \ll \min(C, d_{L-1})$, this approach substantially decreases computational costs while maintaining comparable model performance.

D. Robustness of Post-Training Performance Over Imbalance Factor

To assess the robustness of both the post-training performance and the pseudo-labeling method, we confirm accuracy across different imbalance factors, as shown in Fig. 7. The imbalance factor controls the degree of class imbalance in the pre-training data, allowing us to systematically analyze performance degradation under different imbalance data distributions.

The results indicate that as the imbalance factor increases, both overall accuracy and pseudo-label accuracy exhibit a gradual decline across all datasets. However, the pseudo-labeling method consistently maintains higher accuracy than the overall

TABLE IX
PERFORMANCE COMPARISON OF POST-TRAINING PERFORMANCE IN DYNAMIC SETTING II

Method	NSL-KDD [53]				UNSW-NB15 [55]				ToN-IoT [56]			
	Lin	Squ	Sin	Ber	Lin	Squ	Sin	Ber	Lin	Squ	Sin	Ber
Accuracy [%]												
Base	72.6 $\pm$ 0.16	72.0 $\pm$ 0.14	71.6 $\pm$ 0.17	71.2 $\pm$ 0.14	70.8 $\pm$ 0.13	70.1 $\pm$ 0.14	70.5 $\pm$ 0.13	70.0 $\pm$ 0.14	74.7 $\pm$ 0.16	74.1 $\pm$ 0.15	73.5 $\pm$ 0.14	73.0 $\pm$ 0.15
FTH [35]	80.0 $\pm$ 0.12	79.7 $\pm$ 0.15	79.3 $\pm$ 0.13	78.9 $\pm$ 0.14	77.8 $\pm$ 0.16	77.2 $\pm$ 0.13	76.7 $\pm$ 0.14	76.1 $\pm$ 0.15	80.9 $\pm$ 0.14	80.4 $\pm$ 0.12	79.8 $\pm$ 0.13	79.3 $\pm$ 0.16
FTFWH [35]	81.1 $\pm$ 0.17	80.6 $\pm$ 0.13	80.1 $\pm$ 0.14	79.5 $\pm$ 0.12	78.3 $\pm$ 0.12	77.8 $\pm$ 0.17	77.2 $\pm$ 0.16	76.7 $\pm$ 0.13	82.5 $\pm$ 0.15	81.9 $\pm$ 0.16	81.4 $\pm$ 0.12	80.8 $\pm$ 0.15
UOGD [36]	82.2 $\pm$ 0.17	78.6 $\pm$ 0.14	78.3 $\pm$ 0.12	77.8 $\pm$ 0.15	80.0 $\pm$ 0.14	75.8 $\pm$ 0.16	75.3 $\pm$ 0.12	74.7 $\pm$ 0.17	81.4 $\pm$ 0.16	77.5 $\pm$ 0.14	76.9 $\pm$ 0.15	76.2 $\pm$ 0.17
ATLAS [36]	83.5 $\pm$ 0.14	83.0 $\pm$ 0.17	82.4 $\pm$ 0.15	81.9 $\pm$ 0.12	80.5 $\pm$ 0.16	80.1 $\pm$ 0.14	79.5 $\pm$ 0.16	78.9 $\pm$ 0.14	82.6 $\pm$ 0.15	82.0 $\pm$ 0.17	81.4 $\pm$ 0.13	80.9 $\pm$ 0.15
Ours	89.3$\pm$0.12	88.9$\pm$0.15	88.6$\pm$0.13	88.3$\pm$0.11	87.7$\pm$0.16	87.4$\pm$0.12	87.1$\pm$0.14	86.8$\pm$0.12	91.8$\pm$0.14	91.3$\pm$0.11	91.1$\pm$0.16	90.9$\pm$0.12
Improvement [%]	6.9	7.1	7.5	7.8	8.9	9.1	9.6	10.0	11.2	11.3	11.9	12.4
F1-score [%]												
Base	72.5 $\pm$ 0.16	71.8 $\pm$ 0.14	71.3 $\pm$ 0.17	71.2 $\pm$ 0.14	70.8 $\pm$ 0.13	70.0 $\pm$ 0.14	70.4 $\pm$ 0.13	70.1 $\pm$ 0.14	74.5 $\pm$ 0.16	74.1 $\pm$ 0.15	73.6 $\pm$ 0.14	73.0 $\pm$ 0.15
FTH [35]	79.9 $\pm$ 0.12	79.4 $\pm$ 0.15	79.3 $\pm$ 0.13	78.9 $\pm$ 0.14	77.7 $\pm$ 0.16	77.2 $\pm$ 0.13	76.9 $\pm$ 0.14	76.1 $\pm$ 0.15	81.0 $\pm$ 0.14	80.5 $\pm$ 0.12	79.8 $\pm$ 0.13	79.4 $\pm$ 0.16
FTFWH [35]	80.8 $\pm$ 0.17	80.6 $\pm$ 0.13	79.9 $\pm$ 0.14	79.4 $\pm$ 0.12	78.2 $\pm$ 0.12	78.0 $\pm$ 0.17	77.2 $\pm$ 0.16	76.8 $\pm$ 0.13	82.5 $\pm$ 0.15	81.9 $\pm$ 0.16	81.4 $\pm$ 0.12	80.7 $\pm$ 0.15
UOGD [36]	82.0 $\pm$ 0.17	78.5 $\pm$ 0.14	78.1 $\pm$ 0.12	78.0 $\pm$ 0.15	79.8 $\pm$ 0.14	75.5 $\pm$ 0.16	75.4 $\pm$ 0.12	74.7 $\pm$ 0.17	81.1 $\pm$ 0.16	77.4 $\pm$ 0.14	76.9 $\pm$ 0.15	76.5 $\pm$ 0.17
ATLAS [36]	83.6 $\pm$ 0.14	82.8 $\pm$ 0.17	82.7 $\pm$ 0.15	81.9 $\pm$ 0.12	80.5 $\pm$ 0.16	80.3 $\pm$ 0.14	79.7 $\pm$ 0.16	78.8 $\pm$ 0.14	82.6 $\pm$ 0.15	81.8 $\pm$ 0.17	81.2 $\pm$ 0.13	80.7 $\pm$ 0.15
Ours	89.3$\pm$0.13	88.7$\pm$0.12	88.5$\pm$0.14	88.4$\pm$0.12	87.6$\pm$0.12	87.3$\pm$0.12	86.9$\pm$0.13	86.8$\pm$0.11	92.0$\pm$0.12	91.4$\pm$0.11	91.0$\pm$0.12	91.2$\pm$0.11
Improvement [%]	6.8	7.1	7.0	7.9	8.8	8.7	9.0	10.1	11.4	11.7	12.1	13.0

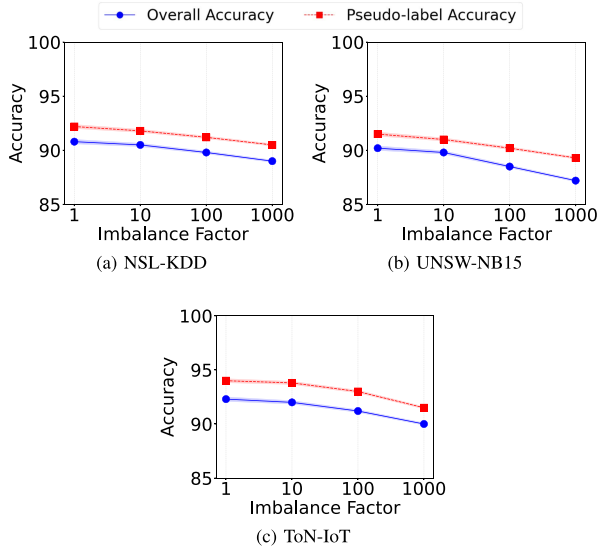


Fig. 7. Robustness of pseudo-label and overall system across imbalance factor of pre-training data under Squ shift.

classification performance, suggesting that the pseudo-labeling process remains reliable even under severe class imbalance. This indicates that while the model experiences performance degradation due to data imbalance, the pseudo-labeling mechanism preserves a relatively stable structure, mitigating excessive performance drops.

Furthermore, the performance gap between pseudo-label accuracy and overall accuracy remains consistent across different datasets, demonstrating the robustness of the proposed pseudo-labeling method. These findings highlight the effectiveness of our approach in handling imbalanced data and ensuring stable learning even under highly skewed distributions. The results for the various shift types are presented in Appendix D, available online.

E. Performance Comparisons

We conducted a performance comparison between state-of-the-art post-training methods and our proposed system. We evaluated performance using multiple datasets, different label shift scenarios, and settings, as presented in Tables VII and IX.

1) *Dynamic Setting I*: This setting is the most popular shift setting in post-training research, where the distribution shifts from a uniform distribution to a Dirac delta function that consists of only a single class [35], [36], [57]. The results show that the proposed method consistently surpasses all baseline approaches, highlighting its adaptability and effectiveness. For all datasets and scenarios, the proposed method achieves the highest accuracy and F1-score across all shift types, with an average improvement of 7.7% over the best-performing baseline in Table VI. This improvement indicates the effectiveness of our strategy in addressing both class imbalance and dynamic shifts.

2) *Dynamic Setting II*: The performance of the proposed method is evaluated in realistic network scenarios, incorporating more complex and non-uniform shifts in label distributions. Despite the increased complexity of these scenarios, our method consistently demonstrates high accuracy, surpassing the baseline models while effectively adapting to variability in the label distributions. For all datasets, the proposed method consistently achieved the highest accuracy and F1-score across all shift types, showing an average improvement of 9.56% over the best-performing baseline. Furthermore, while the best-performing comparison method, ATLAS, experienced a greater performance drop between Dynamic Setting I and Dynamic Setting II, the proposed method showed a comparatively smaller decrease.

Therefore, the proposed approach achieved an average improvement of 8.6% in accuracy and F1-score across the two dynamic settings. This indicates the ability of our system to effectively handle diverse and dynamic conditions.

IX. CONCLUSION

In this work, we propose an efficient post-training system to enhance classification performance in dynamic network intrusion scenarios. The method combines contrastive learning-based pre-training with conditional post-training guided by confidence-based pseudo-labeling and LoRA adaptation. In the pre-training phase, the model incorporates imbalance-aware sampling and focuses on borderline samples, while during deployment, it selectively adapts to new data using confidence signals. This framework effectively enhances the model's adaptability and discriminative power under dynamic and imbalanced conditions, leading to more robust and reliable intrusion detection in real-world network environments. This effectiveness arises because the refined representations learned during pre-training provide a more stable basis for generating reliable pseudo-labels in the post-training stage. Experimental results show that our method consistently outperforms state-of-the-art post-training techniques, with an average improvement of 8.6% in accuracy and F1-score across multiple network datasets. These findings demonstrate the effectiveness of the proposed approach and suggest its practical value for intrusion detection in evolving AIoT environments. It is also worth noting that in settings with ample data and computational resources, full fine-tuning may be a more suitable adaptation strategy than LoRA updates. Future research could explore adaptive confidence mechanisms and cross-domain extensions of the proposed framework to enhance its applicability in large-scale, real-time intrusion detection across diverse network environments.

REFERENCES

- [1] D. Chao, D. Xu, F. Gao, C. Zhang, W. Zhang, and L. Zhu, "A systematic survey on security in anonymity networks: Vulnerabilities, attacks, defenses, and formalization," *IEEE Commun. Surv. Tut.*, vol. 26, no. 3, pp. 1775–1829, thirdquarter 2024.
- [2] Z. Yang, M. Chen, W. Saad, C. S. Hong, and M. Shikh-Bahaei, "Energy efficient federated learning over wireless communication networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 3, pp. 1935–1949, Mar. 2021.
- [3] W. Wu et al., "AI-Native network slicing for 6G networks," *IEEE Wireless Commun.*, vol. 29, no. 1, pp. 96–103, Feb. 2022.
- [4] H. Kye, M. Kim, and M. Kwon, "Hierarchical autoencoder for network intrusion detection," in *Proc. IEEE Int. Conf. Commun.*, May 2022, pp. 2700–2705.
- [5] X. Shen, J. Gao, W. Wu, M. Li, C. Zhou, and W. Zhuang, "Holistic network virtualization and pervasive network intelligence for 6G," *IEEE Commun. Surv. Tut.*, vol. 24, no. 1, pp. 1–30, Firstquarter 2021.
- [6] X. Shen et al., "AI-assisted network-slicing based next-generation wireless networks," *IEEE Open J. Veh. Technol.*, vol. 1, pp. 45–66, 2020.
- [7] M. Joe, M. Kim, and M. Kwon, "Contrastive learning based network attack classifier for imbalanced data," *J. Commun. Netw.*, vol. 28, no. 1, pp. 86–97, Feb. 2026.
- [8] H. Feng et al., "Multi-domain collaborative two-level DDoS detection via hybrid deep learning," *Comput. Netw.*, vol. 242, Apr. 2024, Art. no. 110251.
- [9] M. Kim, M. Joe, and M. Kwon, "Improving network attack classification on imbalanced real-world intrusion incident datasets," in *Proc. ACM Int. Conf. Mobile Sys. Appl. Serv.*, Sep. 2025, pp. 591–592.
- [10] K. Miao, M. Zhang, F. Guo, R. Lu, and X. Guan, "Detection of false data injection attacks in smart grids: An optimal transport-based reliable self-training approach," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 709–723, 2025.
- [11] H. Park, M. Kim, and M. Kwon, "Personalized federated sensing for heterogeneous environment," *IEEE Sensors Lett.*, vol. 9, no. 4, Apr. 2025, Art. no. 6004004.
- [12] O. Al-Jarrah, O. Alhussein, P. Yoo, S. Muhaidat, K. Taha, and K. Kim, "Data randomization and cluster-based partitioning for botnet intrusion detection," *IEEE Trans. Cybern.*, vol. 46, no. 8, pp. 1796–1806, Aug. 2016.
- [13] H. Kye and M. Kwon, "Partial federated learning based network intrusion system for mobile devices," in *Proc. ACM Int. Symp. Mobile Ad Hoc Netw. Comput.*, Oct. 2022, pp. 283–284.
- [14] Z. Almahmoud, P. Yoo, O. Alhussein, I. Farhat, and E. Damiani, "A holistic and proactive approach to forecasting cyber threats," *Sci. Rep.*, vol. 13, no. 1, May 2023, Art. no. 8049.
- [15] M. Dib, S. Torabi, E. Bou-Harb, N. Bouguila, and C. Assi, "EVOLIoT: A self-supervised contrastive learning framework for detecting and characterizing evolving IoT malware variants," in *Proc. ACM Comput. Commun. Secur.*, May 2022, pp. 452–466.
- [16] Y. Yue, X. Chen, Z. Han, X. Zeng, and Y. Zhu, "Contrastive learning enhanced intrusion detection," *IEEE Trans. Netw. Serv. Manage.*, vol. 19, no. 4, pp. 4232–4247, Dec. 2022.
- [17] Y. Jiao, K. Yang, D. Song, and D. Tao, "TimeAutoAD: Autonomous anomaly detection with self-supervised contrastive loss for multivariate time series," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 3, pp. 1604–1619, May/Jun. 2022.
- [18] D. Elreedy and A. Atiya, "A comprehensive analysis of synthetic minority oversampling technique (SMOTE) for handling class imbalance," *Inf. Sci.*, vol. 505, pp. 32–64, Dec. 2019.
- [19] M. Kim, M. Joe, and M. Kwon, "OASIS: Open-world adaptive self-supervised and imbalanced-aware system," in *Proc. 34th ACM Int. Conf. Inf. Knowl. Manage.*, Nov. 2025, pp. 1375–1385.
- [20] H. Park, M. Joe, M. Kim, and M. Kwon, "ASAP: Unsupervised post-training with label distribution shift adaptive learning rate," in *Proc. 34th ACM Int. Conf. Inf. Knowl. Manage.*, 2025, pp. 5094–5098.
- [21] M. Kim, H. Park, and M. Kwon, "Personalized split federated learning with early-exit: Pre-training and online learning against label shifts," *IEEE Internet Things J.*, vol. 12, no. 22, Nov. 2025.
- [22] S. Amalapuram, B. Tamma, and S. Channappayya, "SPIDER: A semi-supervised continual learning-based network intrusion detection system," in *Proc. IEEE Conf. Comput. Commun.*, May 2024, pp. 571–580.
- [23] Z. Yang et al., "MLoRA: Multi-domain low-rank adaptive network for CTR prediction," in *Proc. 18th ACM Conf. Recommender Syst.*, Oct. 2024, pp. 287–297.
- [24] A. Agiza, M. Neseem, and S. Reda, "MTLoRA: Low-rank adaptation approach for efficient multi-task learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2024, pp. 16196–16205.
- [25] T. Mehedi, A. Anwar, Z. Rahman, K. Ahmed, and R. Islam, "Dependable intrusion detection system for IoT: A deep transfer learning based approach," *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 1006–1017, Jan. 2023.
- [26] Z. Liu, D. Ye, L. Tang, Y. Zhang, J. Deng, and W. Kuang, "TEAM: Temporal adversarial examples attack model against network intrusion detection system applied to RNN," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 4, pp. 3400–3415, Jul./Aug. 2025.
- [27] J. Leevy, T. Khoshgoftar, R. Bauder, and N. Seliya, "A survey on addressing high-class imbalance in Big Data," *J. Big Data*, vol. 5, no. 1, pp. 1–30, Jan. 2018.
- [28] A. Fernandez, S. Garcia, F. Herrera, and N. Chawla, "SMOTE for learning from imbalanced data: Progress and challenges, marking the 15-year anniversary," *J. Artif. Intell. Res.*, vol. 61, pp. 863–905, Jan. 2018.
- [29] E. Elyan, F. Moreno, and C. Jayne, "CDSMOTE: Class decomposition and synthetic minority class oversampling technique for imbalanced-data classification," *Neural Comput. Appl.*, vol. 33, pp. 2839–2851, Apr. 2021.
- [30] S. Prakosa, M. Faisal, and J. Leu, "Using optimized focal loss for imbalanced dataset on network intrusion detection system," in *Proc. IEEE Veh. Technol. Conf.*, Jun. 2022, pp. 1–7.
- [31] X. Liu et al., "Self-supervised learning: Generative or contrastive," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 857–876, Jan. 2023.
- [32] W. Ma, X. Wang, J. Dong, M. Hu, and Q. Zhou, "A lightweight method for botnet detection in Internet of Things environment," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 4, pp. 2458–2427, Jul./Aug. 2025.
- [33] D. Lee, S. Kim, I. Kim, Y. Cheon, M. Cho, and W. Han, "Contrastive regularization for semi-supervised learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022.
- [34] Y. Yang et al., "ASTREAM: Data-stream-driven scalable anomaly detection with accuracy guarantee in IIoT environment," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 3007–3016, Sep./Oct. 2023.
- [35] R. Wu, C. Guo, Y. Su, and K. Weinberger, "Online adaptation to label distribution shift," in *Proc. Adv. Neural Inf. Process. Syst.*, Dec. 2021, pp. 11340–11351.

- [36] Y. Bai, Y. Zhang, P. Zhao, M. Sugiyama, and Z. Zhou, "Adapting to online label shift with provable guarantees," in *Proc. Adv. Neural Inf. Process. Syst.*, Dec. 2022, pp. 29960–29974.
- [37] Y. Liang and W. Li, "InfLoRA: Interference-free low-rank adaptation for continual learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2024, pp. 23638–23647.
- [38] S. He, Y. Lei, Y. Zhang, K. Xie, and P. Sharma, "Parameter-efficient log anomaly detection based on pre-training model and LoRA," in *Proc. IEEE Int. Symp. Softw. Rel. Eng.*, Nov. 2023, pp. 207–217.
- [39] Z. Zhou, L. Guo, L. Jia, D. Zhang, and Y. Li, "ODS: Test-time adaptation in the presence of open-world data shift," in *Proc. Int. Conf. Mach. Learn.*, Jul. 2023, pp. 42574–42588.
- [40] J. Xu and S. Huang, "A joint training-calibration framework for test-time personalization with label shift in federated learning," in *Proc. 32nd ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2023, pp. 4370–4374.
- [41] M. Wang et al., "Joint adversarial domain adaptation with structural graph alignment," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 1, pp. 604–612, Jan./Feb. 2024.
- [42] G. Prasad, A. Pandey, and S. Kumar, "Domain adaptation for localization using combined autoencoder and gradient reversal layer in dynamic IoT environment," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 1, pp. 685–695, Jan./Feb. 2024.
- [43] V. Silva, L. Carvalho, J. Lloret, and M. Proença, "f-AnoGAN for unsupervised attack detection in SDN environment," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 4, pp. 3271–3285, Jul./Aug. 2025.
- [44] M. Hashemi and E. Keller, "General domain adaptation through proportional progressive pseudo labeling," in *Proc. IEEE Int. Conf. Big Data*, Dec. 2020, pp. 155–162.
- [45] H. Edward et al., "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Representations*, Apr. 2022.
- [46] H. Kye, M. Kim, and M. Kwon, "Hierarchical detection of network anomalies: A self-supervised learning approach," *IEEE Signal Process. Lett.*, vol. 29, pp. 1908–1912, 2022.
- [47] Y. Liu, Y. Gu, X. Shen, Q. Liao, and Q. Yu, "MSCA: An unsupervised anomaly detection system for network security in backbone network," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 1, pp. 223–238, Jan./Feb. 2023.
- [48] Z. Ding and Y. Fu, "Deep transfer low-rank coding for cross-domain learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 6, pp. 1768–1779, Jun. 2019.
- [49] W. Zhao, C. Xu, Z. Guan, and Y. Liu, "Multiview concept learning via deep matrix factorization," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 2, pp. 814–825, Feb. 2021.
- [50] W. Zhang, J. Wu, W. Zhao, H. Deng, and Y. Yang, "Hierarchical one-class model with subnetwork for representation learning and outlier detection," *IEEE Trans. Cybern.*, vol. 53, no. 10, pp. 6303–6316, Oct. 2023.
- [51] J. Kim, J. Kim, and E. K. Ryu, "LoRA training provably converges to a low-rank global minimum or it fails loudly (but it probably won't fail)," in *Proc. Int. Conf. Mach. Learn.*, Jul. 2025.
- [52] M. Eskandar, T. Imtiaz, D. Hill, Z. Wang, and J. Dy, "Star: Stability-inducing weight perturbation for continual learning," in *Proc. Int. Conf. Learn. Representations*, Apr. 2025.
- [53] Y. Ding and Y. Zhai, "Intrusion detection system for NSL-KDD dataset using convolutional neural networks," in *Proc. Int. Conf. Comput. Sci. Artif. Intell.*, Dec. 2018, pp. 81–85.
- [54] M. Tavallaei, E. Bagheri, W. Lu, and A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl.*, Dec. 2009, pp. 1–6.
- [55] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. IEEE Mil. Commun. Inf. Syst. Conf.*, Oct. 2015, pp. 1–6.
- [56] A. Sharma, S. Rani, S. Shah, R. Sharma, F. Yu, and M. Hassan, "An efficient hybrid deep learning model for denial of service detection in cyber physical systems," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2419–2428, Sep./Oct. 2023.
- [57] C. Ye, R. Tsuchida, L. Petersson, and N. Barnes, "Label shift estimation for class-imbalance problem: A Bayesian approach," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, Jan. 2024, pp. 1073–1082.



and online learning techniques that address data heterogeneity, resource constraints, and dynamic distribution shifts. He was the recipient of the AI SeoulTech Graduate Scholarship from the Seoul Scholarship Foundation and the Best Paper Award from the HFR Paper Competition.



Research Foundation of Korea (NRF) and the Best Paper Award from the HFR Paper Competition.



Minhae Kwon (Senior Member IEEE) received the B.S., M.S., and Ph.D. degrees in 2011, 2013, and 2017, respectively, from Ewha Womans University, Seoul, Republic of Korea. She was a Postdoctoral Researcher with the Department of Electrical and Computer Engineering with Rice University and was also with the Department of Neuroscience, Baylor College of Medicine, Houston, TX, USA. She was an Associate Professor with Soongsil University. She is currently an Associate Professor with the Department of Electrical and Computer Engineering, Sungkyunkwan University (SKKU), Suwon, Republic of Korea. She was the recipient of the Minister's Award and the Excellence Award for Young Scientists from the Korean Government, Global Ph.D. Fellowship, Women Technicians Fellowship from Google Inc., Qualcomm Innovation Award from Qualcomm Inc., Best Paper Award from the IEEE Consumer Electronics Society, and the KICS Academic Excellence Award and the KICS Haedong Best Paper Award.